

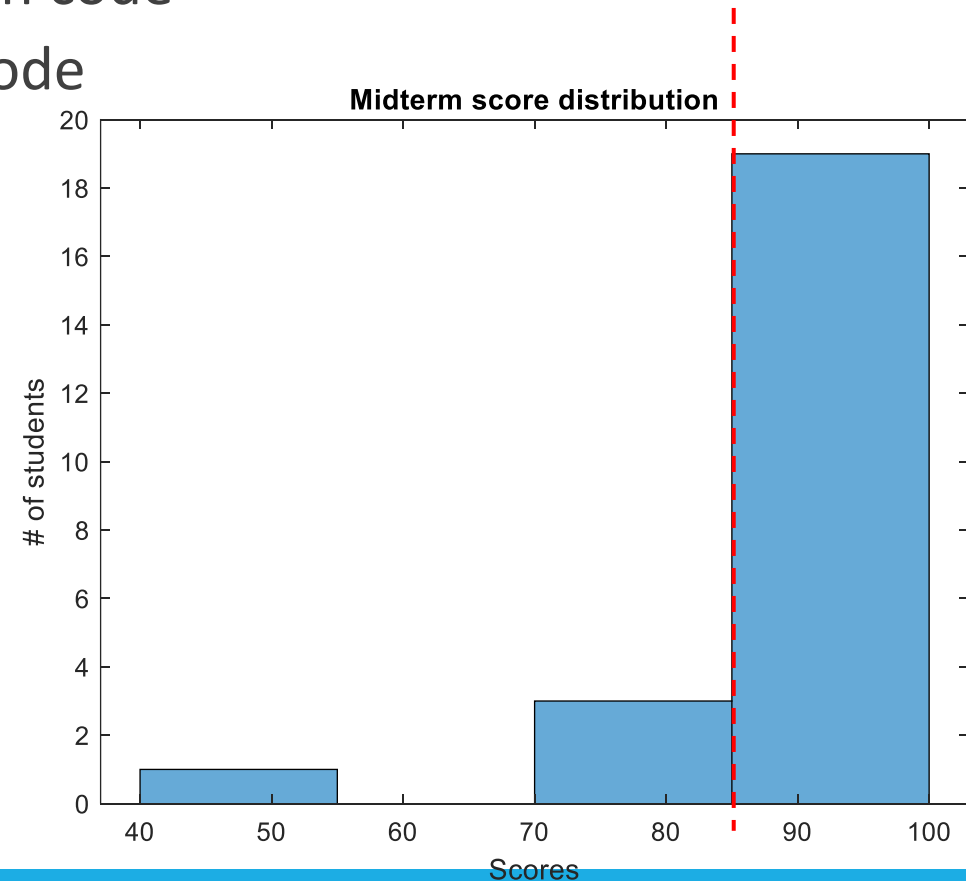
Deep Learning and Deep Reinforcement Learning

KUO-SHIH TSENG
DEPARTMENT OF MATHEMATICS
NATIONAL CENTRAL UNIVERSITY, TAIWAN

2025/10/13

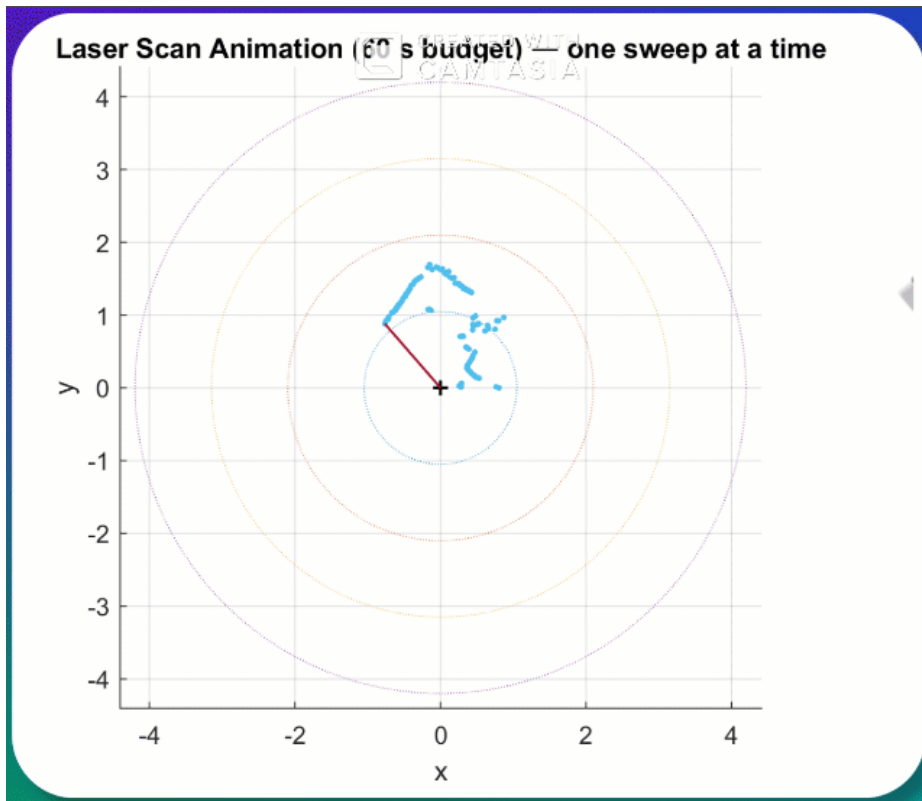
Course Announcement

- HW1:
 - Collect Laser data (25%) - via C/C++ or Python
 - Collect IMU data (25%) - via Python with code
 - Bayes filter (50%) – math and Matlab code
- <80 : **4/23**
- 80~100: 19/23
- Average: **89.8**

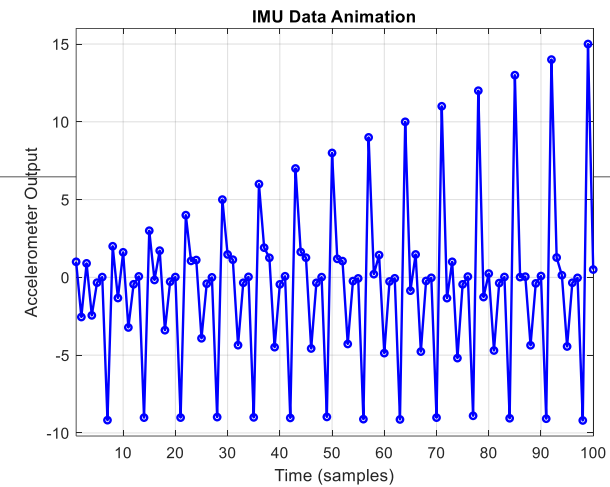


Course Announcement

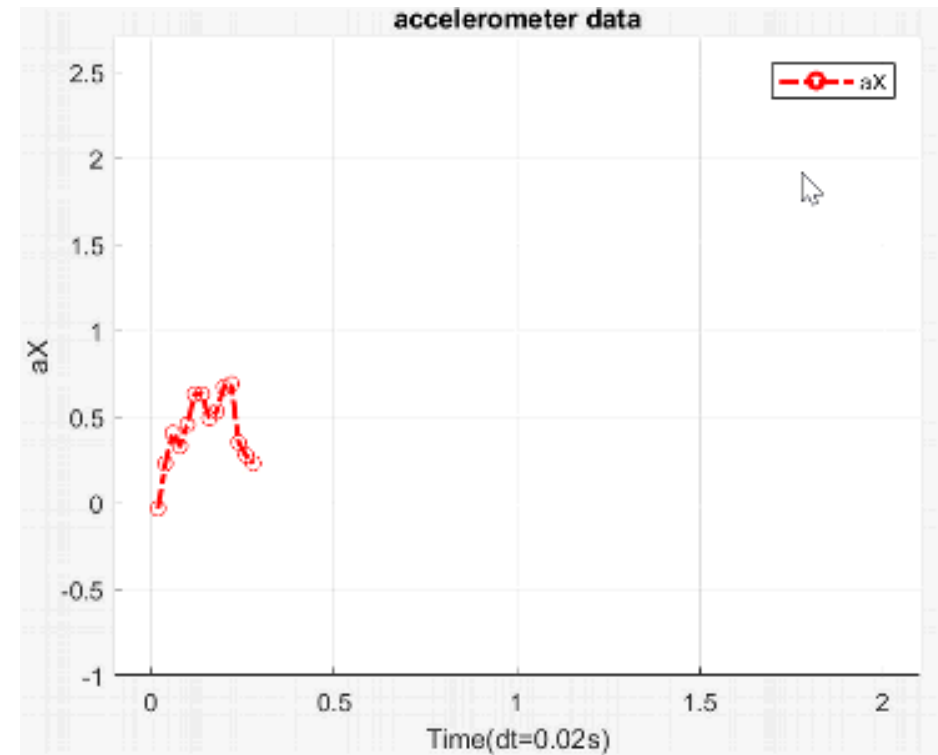
- HW1
 - Collect Laser data (25%)
 - Collect IMU data (25%)



Implemented by 張景程



@@"

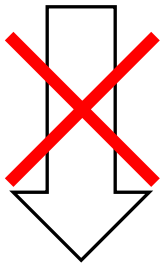


Implemented by 高逸慈

Course Announcement

- HW1
 - Bayes filter (50%)

$$Z = \begin{bmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{bmatrix}$$



MATLAB
CODE

```
Z = [ 0.9, 0.1;
      0.3, 0.7 ] ;
```

Let the state variable $X_t \in \{1, \dots, n\}$

The transition model $P(X_t | X_{t-1})$ is a $n \times n$ matrix $\mathbf{T}$,
where $\mathbf{T}_{ij} = P(X_t = j | X_{t-1} = i)$

The sensor model $P(z_t | X_t)$ is a $n \times n$ matrix $\mathbf{O}_t$,
whose i th diagonal entry is $P(z_t | X_t = i)$ and others are 0.

$$\mathbf{T} = P(X_t | X_{t-1}) = \begin{bmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{bmatrix}$$

$$\mathbf{O}_1 = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.2 \end{bmatrix}, \mathbf{O}_3 = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.8 \end{bmatrix}$$

```
filter_Z1 = 0.1486  0.1026  0.0976  0.6929  0.8230  0.8461  0.8500  0.2134  0.1101  0.0984
```

Course Announcement

- HW1
- Use “clear;clc;close all” in each .m file. → keep the same memory setting.
- “Error using assert 找不到檔案：C:\Users\99861\OneDrive\文件\DS_HW1\laser.dat” → Please set the data path.
- If you have computer issues, please call for help from MRL instead of Math department office since you need computers with Linux & VM.
- **Late policy:** If your HW/project is late for 1 day, the discount rate is **0.8**. For 2 days, the discount rate is 0.8^2 . and so on.

Course Announcement

- You have 3 weeks for HW. Try to do it ASAP.



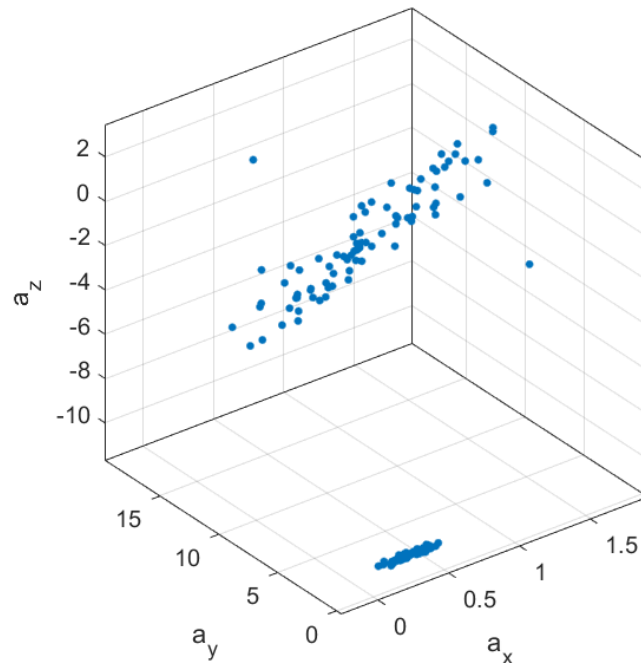
中國網路瘋玩「王世堅 A I」梗 政治迷因逆滲透掀模仿潮

Course Announcement

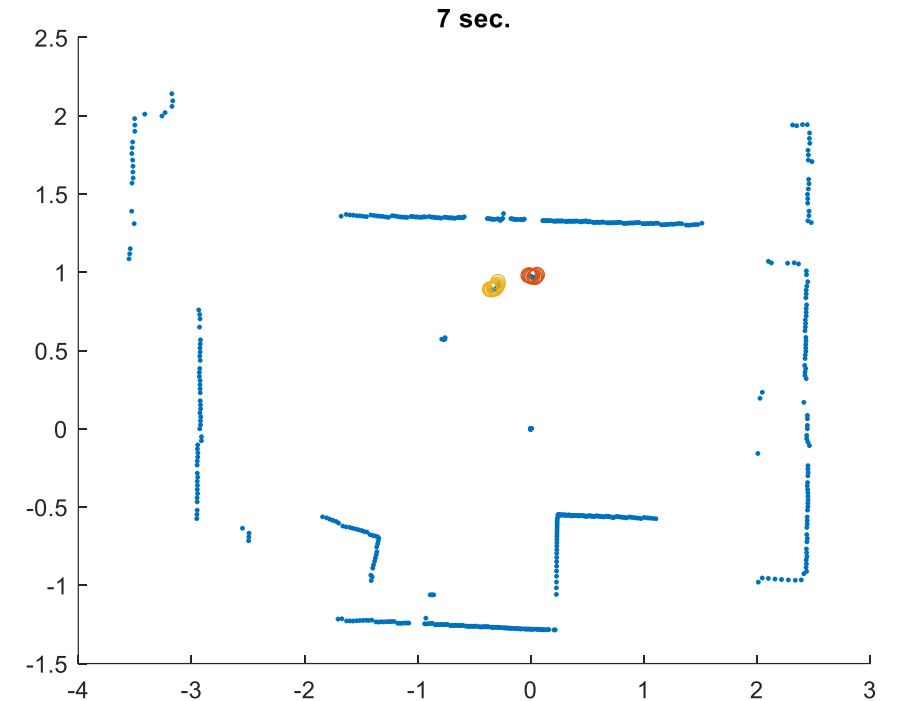
- HW2 was released (Deadline: 10/20(Mon.) 00:00am):
 - Bayesian inference (40%)
 - Supervised learning for IMU data (20%)
 - Supervised learning for Laser data (40%)



Sky Bow missile



3D IMU data



5D leg data

Course Announcement

Week	Date	Topic	Remark	
1	9/1/2025	L1: Introudction	L2: Bayes theorem	
2	9/8/2025	L3: Bayes theorem over time	HW1	
3	9/15/2025	ROS tutorial		
4	9/22/2025	L4: Naïve Bayes and Percept		
5	9/29/2025	L5: Adaboost	HW1 DL, HW2	
6	10/6/2025	Holiday		
7	10/13/2025	L6: Deep learning		
8	10/20/2025	L7: K-means and EM	HW2 DL	
9	10/27/2025	Midterm		
10	11/3/2025	L8: Kalman filter	HW3	
11	11/10/2025	L9: Extended Kalman fitler		Proposal
12	11/17/2025	L10: Tracking		
13	11/24/2025	L11: 2D localization	HW3 DL	Proposal DL
14	12/1/2025	L12: Mapping		
15	12/8/2025	L13: SLAM		
16	12/15/2025	Final Project presentation & Demo		

Course Announcement

- Midterm: Robot Challenge, 10/27(Mon.) 8am-11am
- Team: **2** students in a group, give me your partner name by 10/20(Mon.)
- You must implement it on ROS via *C++* or *Python* with sample code
- Detect and estimate for 2 objects (classifier + Bayes filter)
 1. Ball
 2. Box
 3. Moving ball
- The output should indicate
 - (object_x, object_y, probability, object_index1)
 - (object_x, object_y, probability, object_index2)
- Scores: 0, 40, 60, 80 (probability+ 20).
- **Bonus**: display 3D locations on rviz +20!
- Rosbag for saving data.

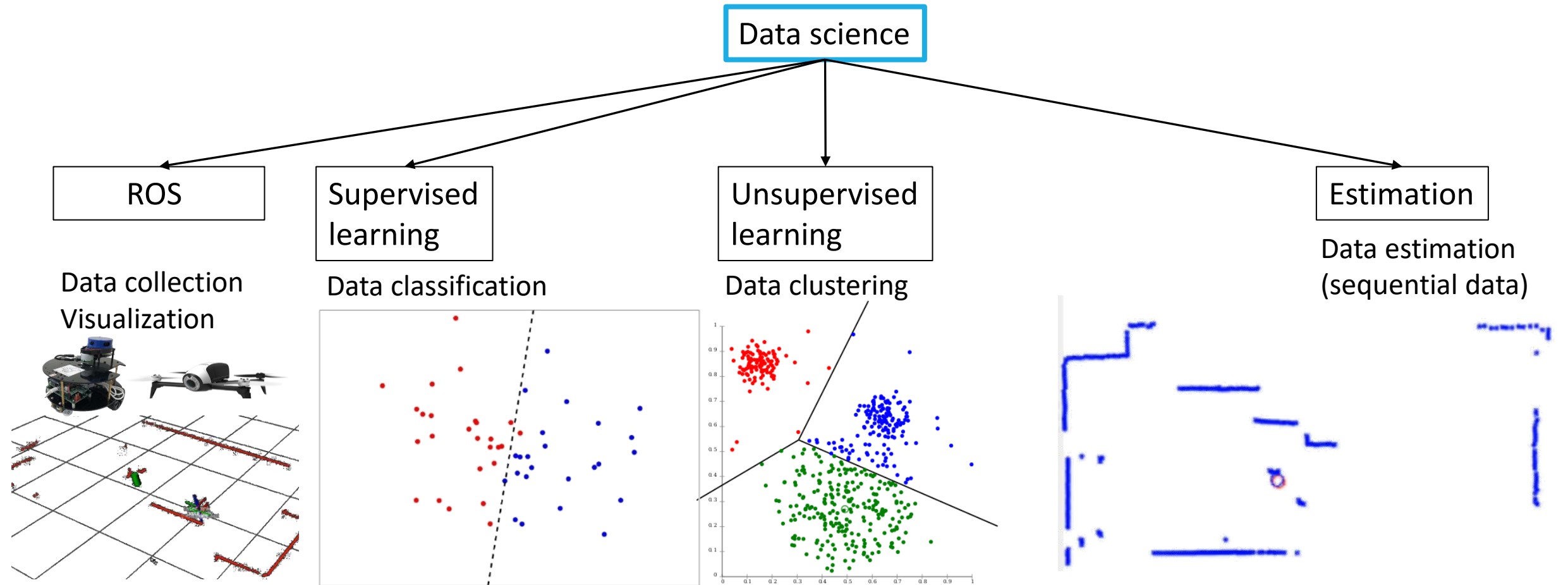
2021 Midterm video: <https://youtu.be/q63ppZKZCAs?t=73>



Outline

- Neural Network
- Multilayer Perceptron
- Weighted Filter
- Convolution Neural Network
- EX: MNIST
- Deep Reinforcement Learning
- EX: Atari with DQN
- EX: Turtlebot3 with DQN

Outline

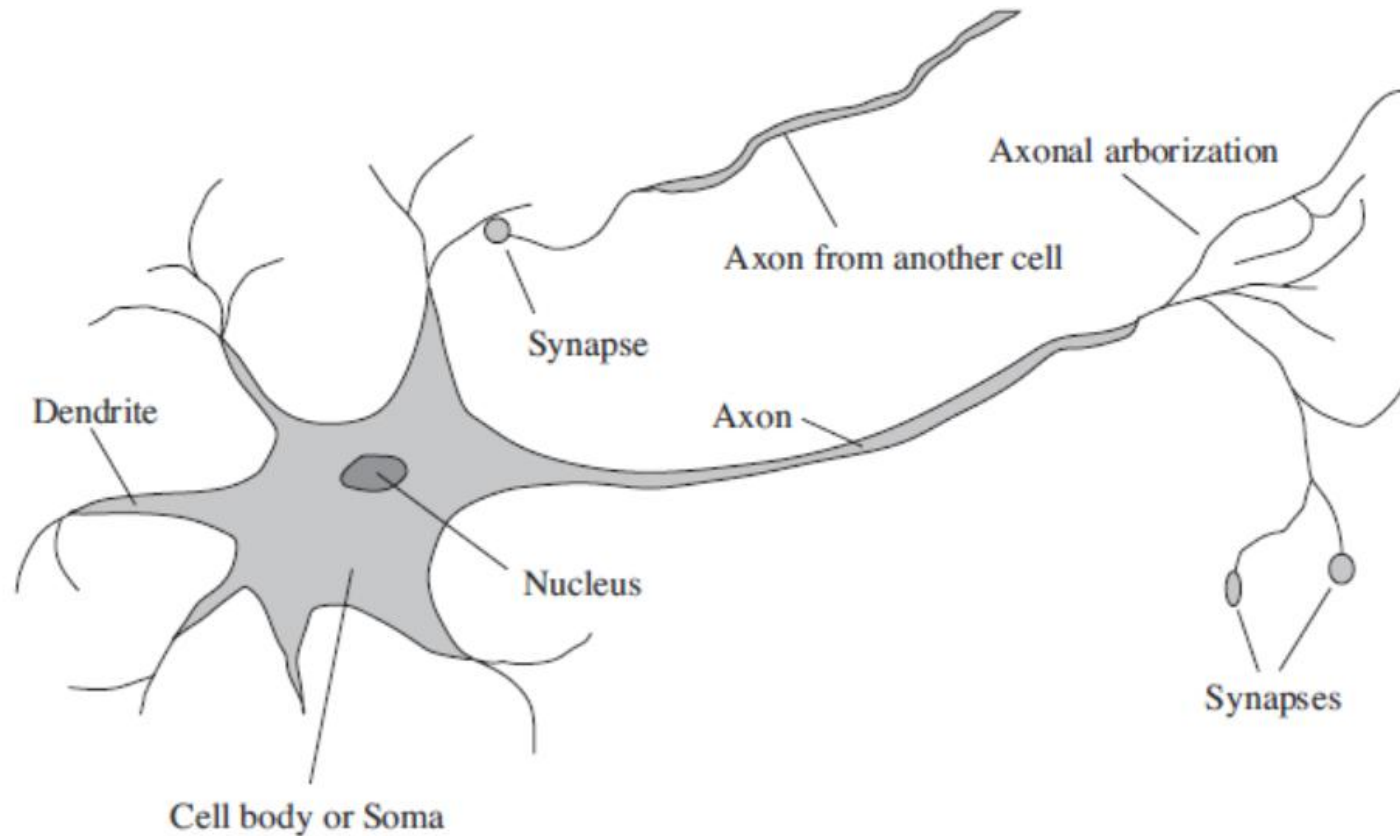


Need

- Belief of Neural Networks
- <https://www.youtube.com/watch?v=I9RWTMNnvi4>

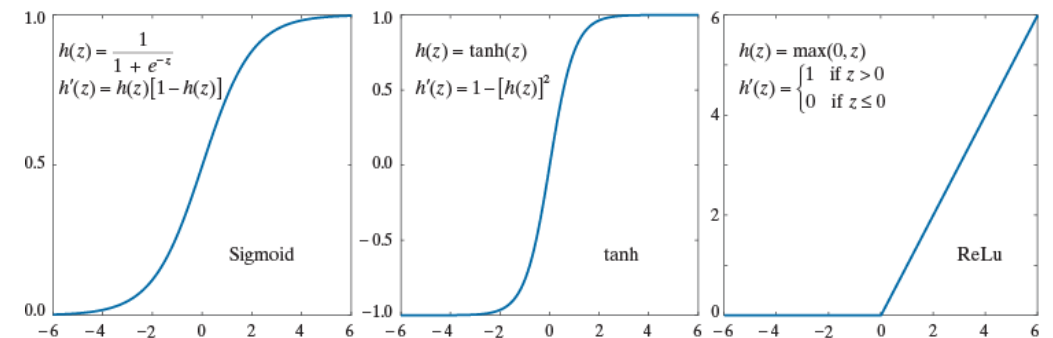
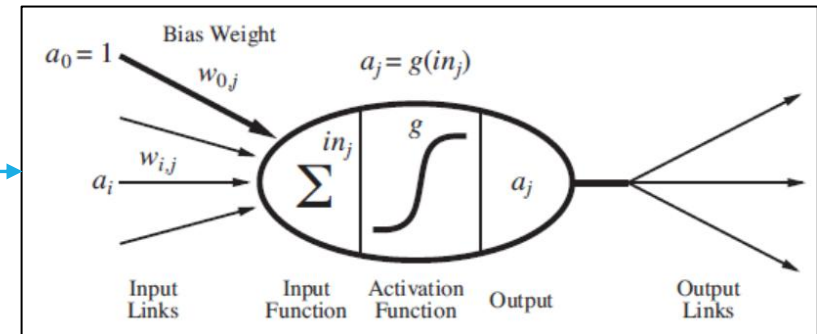
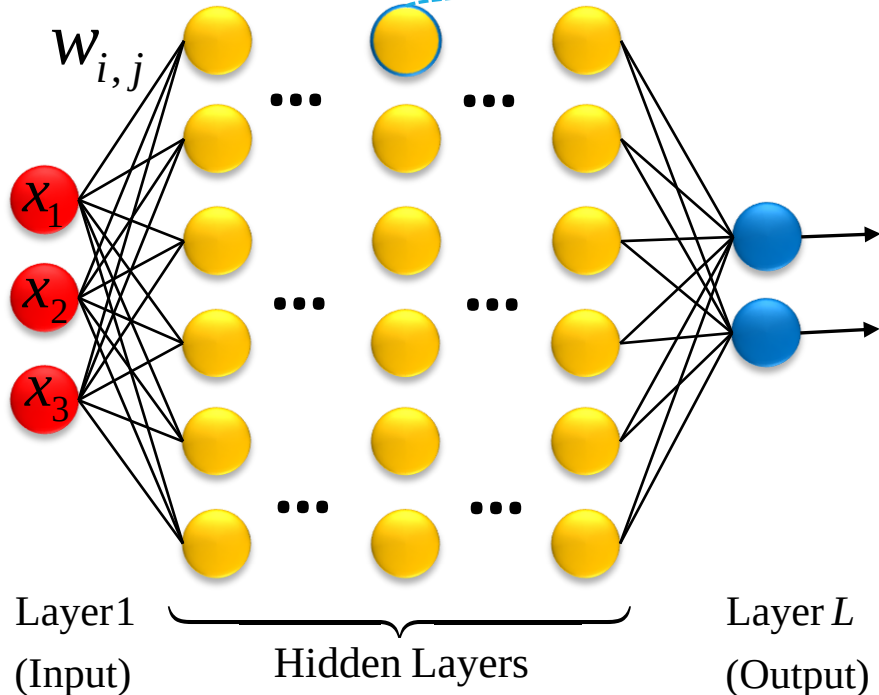
Neural networks

- The concept of neural network (NN) is to simulate human brains' neuron.



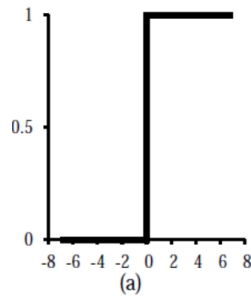
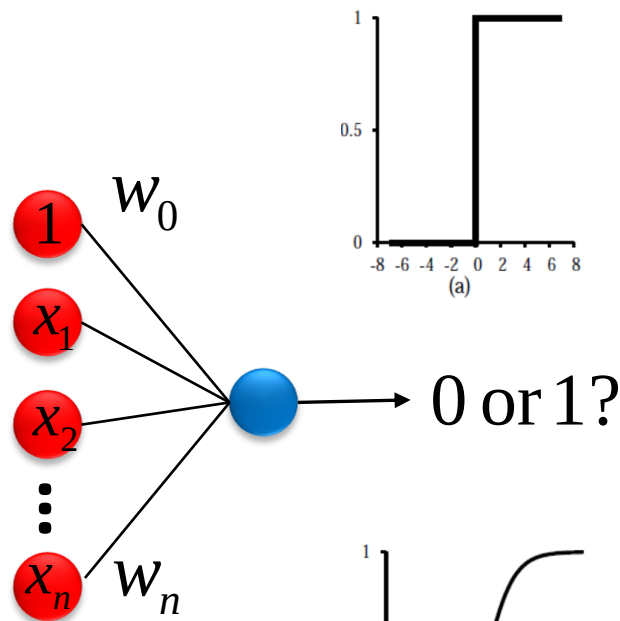
Neural networks

- AI researchers use computers to simulate neuros. Given the input, the NN should automatically tune its weighting for desired output. (Connectionism/bionicsism)

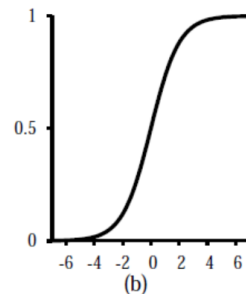


Neural networks

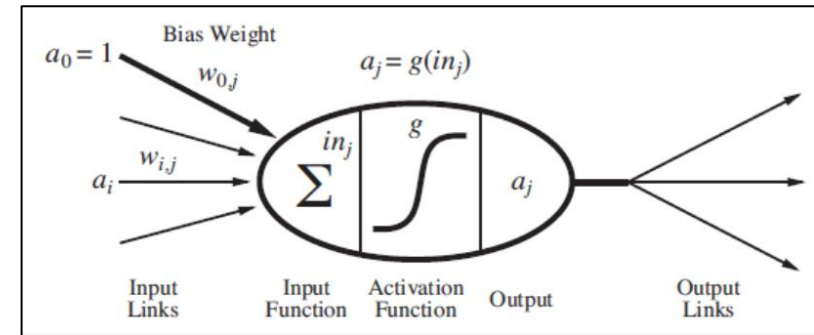
- Recall the special case of NN, a single-layer NN (no hidden layer).
- Perceptron and logistic regression are single-layer NNs.



$$h_W(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}\mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}\mathbf{x} < 0 \end{cases}$$

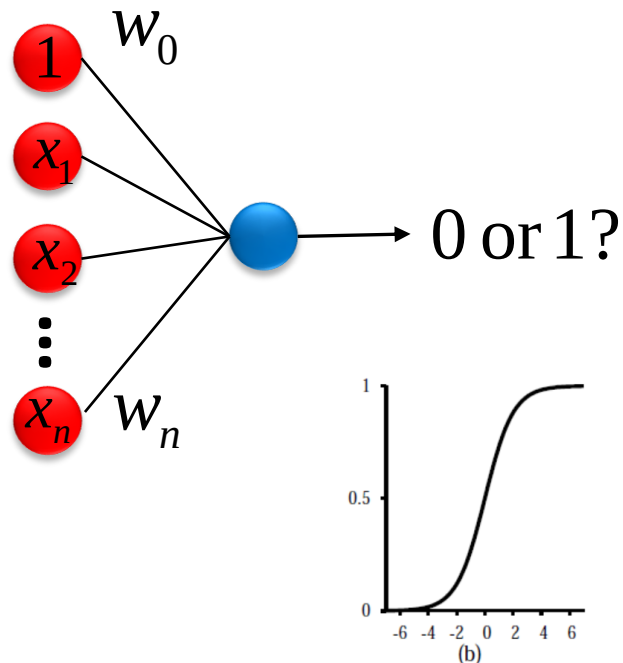


$$h(x) = \text{logistic}(\mathbf{w}\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}\mathbf{x}}}$$



Neural networks

- There are two steps of NN.
 - Prediction (forward propagation): Predict the output based on weighting, input data and NN structure.
 - Update (backward propagation): Update the weighting based on the output labels, input data and NN structure.



$$h(x) = \text{logistic}(\mathbf{w}\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}\mathbf{x}}}$$

Forward propagation

$$w_j \leftarrow w_j + \alpha(y - h_w(\mathbf{x}))h_w(\mathbf{x})(1 - h_w(\mathbf{x}))x_j$$

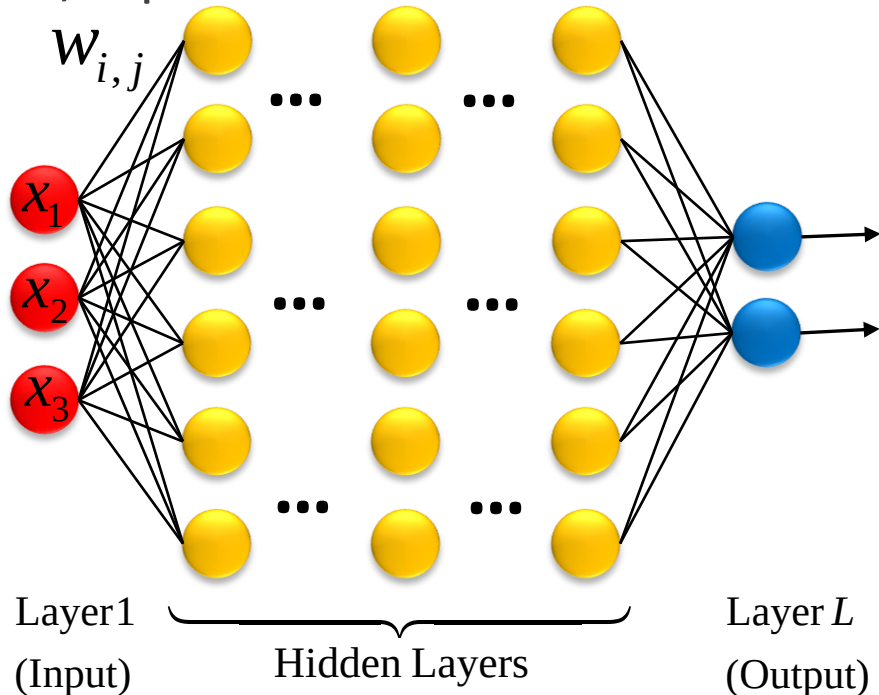
Backward propagation

You learned this in “Numerical Linear Algebra”

https://hackmd.io/@Mes/mes_note/

Neural networks

- There are two steps of NN.
 - Prediction (forward propagation): Predict the output based on weighting, input data and NN structure.
 - Update (backward propagation): Update the weighting based on the output labels, input data and NN structure.



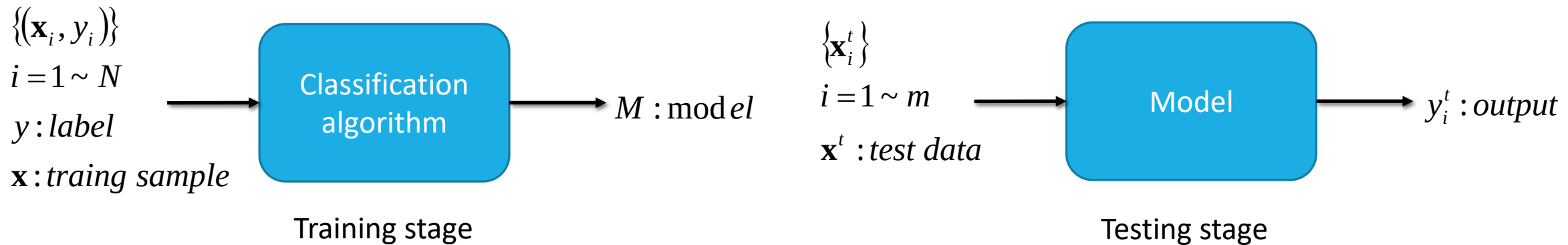
Forward and backward propagations are easy for a single layer NN. How about multilayer NN?

Outline

- Neural Network
- **Multilayer Perceptron**
- Weighted Filter
- Convolution Neural Network
- EX: MNIST
- Deep Reinforcement Learning
- EX: Atari with DQN
- EX: Turtlebot3 with DQN

Multilayer Perceptron (MLP)

- Prediction (forward propagation): Predict the output based on weighting, input data and NN structure. → Testing
- Update (backward propagation): Update the weighting based on the output labels, input data and NN structure. → Training



Multilayer Perceptron (MLP)

Assume there is a 3-layer NN (as Fig. shows).

The activation function of each node is logistic function

Find the NN output?

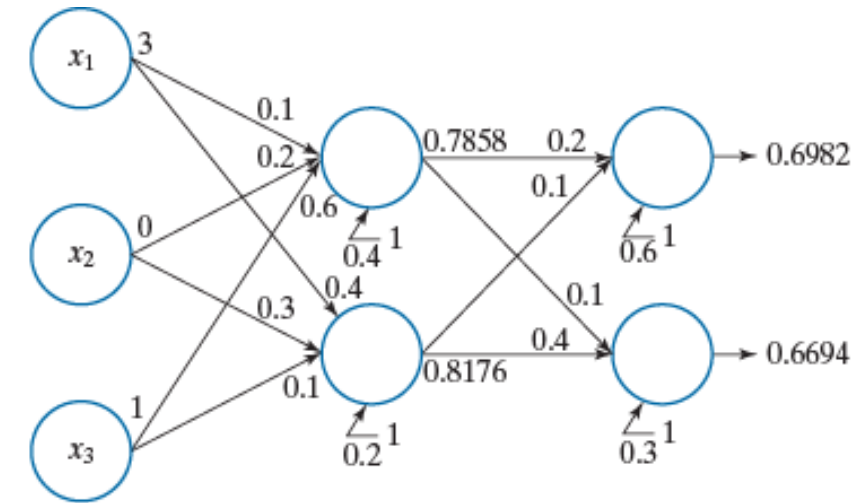
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}$$

$$z_1(2) = W_{1,1}X = (0.1)(3) + (0.2)(0) + (0.6)(1) + (1)(0.4) = 1.3$$

$$a_1(2) = h(z_1(2)) = \frac{1}{1 + e^{-1.3}} = 0.7858$$

$$z_2(2) = W_{1,2}X = (0.4)(3) + (0.3)(0) + (0.1)(1) + (1)(0.2) = 1.5$$

$$a_2(2) = h(z_2(2)) = \frac{1}{1 + e^{-1.5}} = 0.8176$$



Multilayer Perceptron (MLP)

Assume there is a 3-layer NN (as Fig. shows).
The activation function of each node is logistic function
Find the NN output?

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}$$

+bias

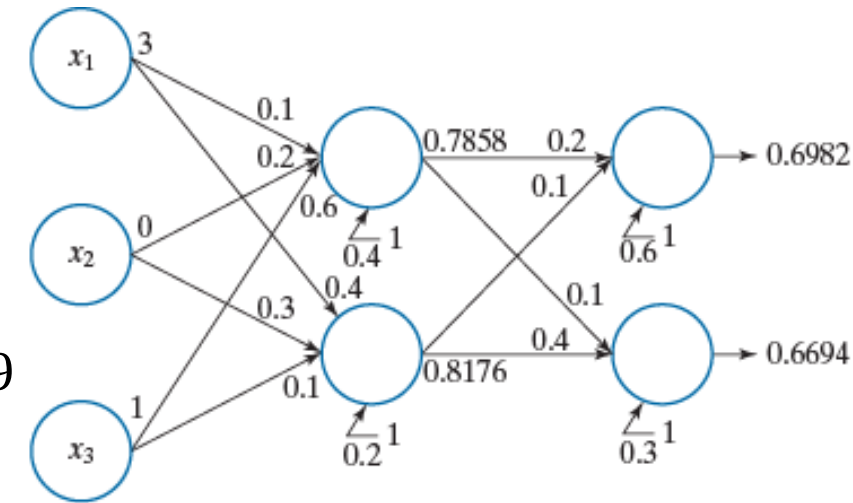
$$z_1(3) = W_{2,1}A(2) = (0.2)(0.7858) + (0.1)(0.8176) + (0.6)(1) = 0.8389$$

$$a_1(3) = h(z_1(3)) = \frac{1}{1 + e^{-0.8389}} = 0.6982$$

$$z_2(3) = W_{2,2}A(2) = (0.1)(0.7858) + (0.4)(0.8176) + (0.3)(1) = 0.7056$$

$$a_2(3) = h(z_2(3)) = \frac{1}{1 + e^{-0.7056}} = 0.6694$$

$\therefore a_1(3) > a_2(3), \therefore$ the data is classified as c_1



Multilayer Perceptron (MLP)

Assume there is a 3-layer NN (as Fig. shows).
 The activation function of each node is logistic function
 Find the NN output?

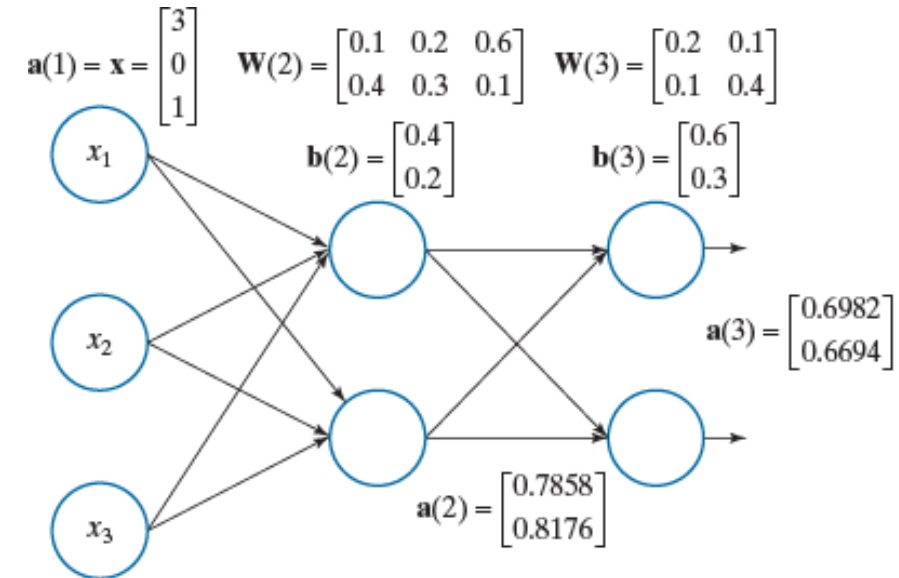
$$a(1) = [3 \quad 0 \quad 1]^T$$

$$z(2) = W(2)a(1) + b(2) = \begin{bmatrix} 0.1 & 0.2 & 0.6 \\ 0.4 & 0.3 & 0.1 \end{bmatrix} \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0.4 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1.3 \\ 1.5 \end{bmatrix}$$

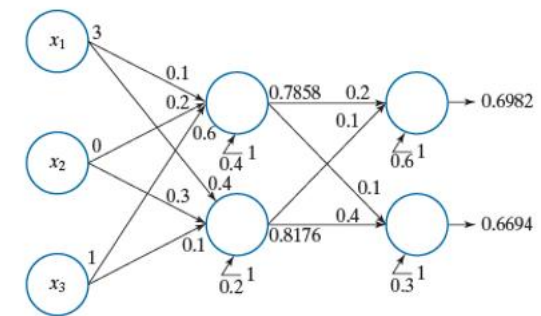
$$a(2) = h[z(2)] = \begin{bmatrix} h(1.3) \\ h(1.5) \end{bmatrix} = \begin{bmatrix} 0.7858 \\ 0.8176 \end{bmatrix}$$

$$z(3) = W(3)a(2) + b(3) = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.4 \end{bmatrix} \begin{bmatrix} 0.7858 \\ 0.8176 \end{bmatrix} + \begin{bmatrix} 0.6 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.8389 \\ 0.7056 \end{bmatrix}$$

$$a(3) = h[z(3)] = \begin{bmatrix} h(0.8389) \\ h(0.7056) \end{bmatrix} = \begin{bmatrix} 0.6982 \\ 0.6694 \end{bmatrix}$$



Matrix form

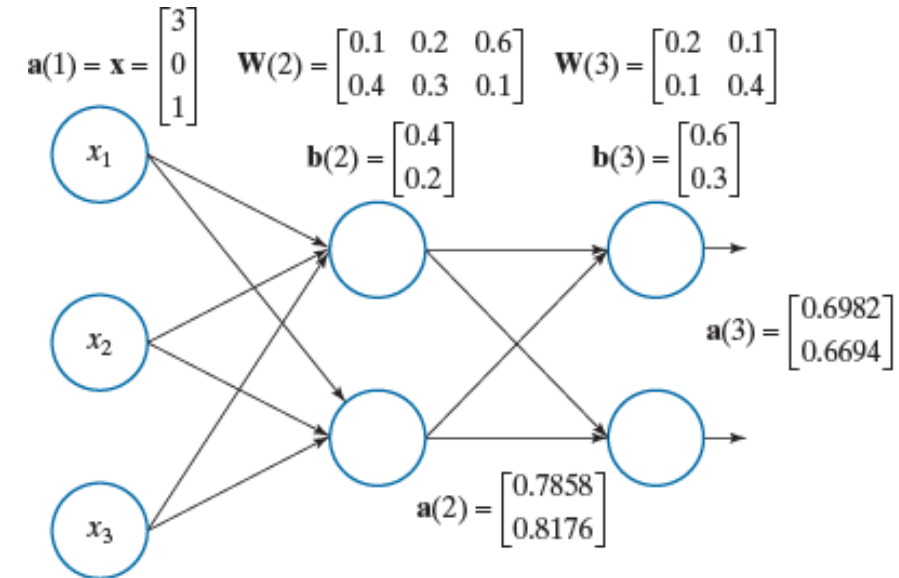


Multilayer Perceptron (MLP)

- Tensorflow code

```
X = tf.Variable([[3.0,0.0,1.0]])  
W = tf.Variable([[0.1,0.4 ],  
                [0.2, 0.3 ],  
                [0.6, 0.1 ]])  
b = tf.Variable([[0.4,0.2]])  
  
XWb=tf.matmul(X,W)+b  
y=tf.nn.sigmoid(tf.matmul(X,W)+b)
```

```
with tf.Session() as sess:  
    init = tf.global_variables_initializer()  
    sess.run(init)  
    print('XWb:')  
    print(sess.run(XWb))  
    print('y:')  
    print(sess.run(y ))
```



Multilayer Perceptron (MLP)

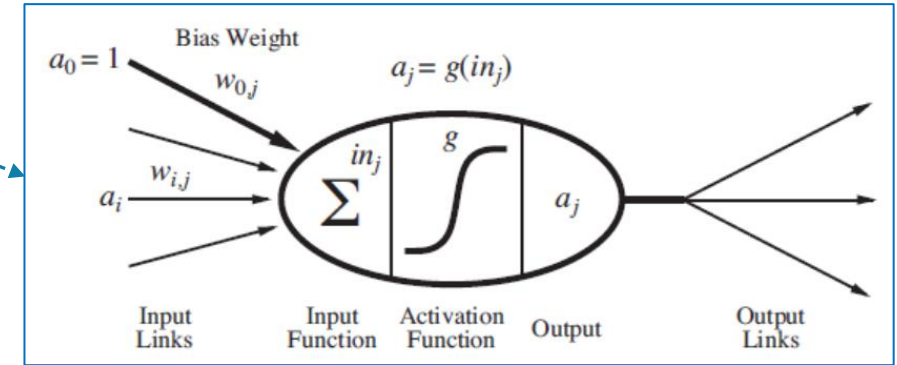
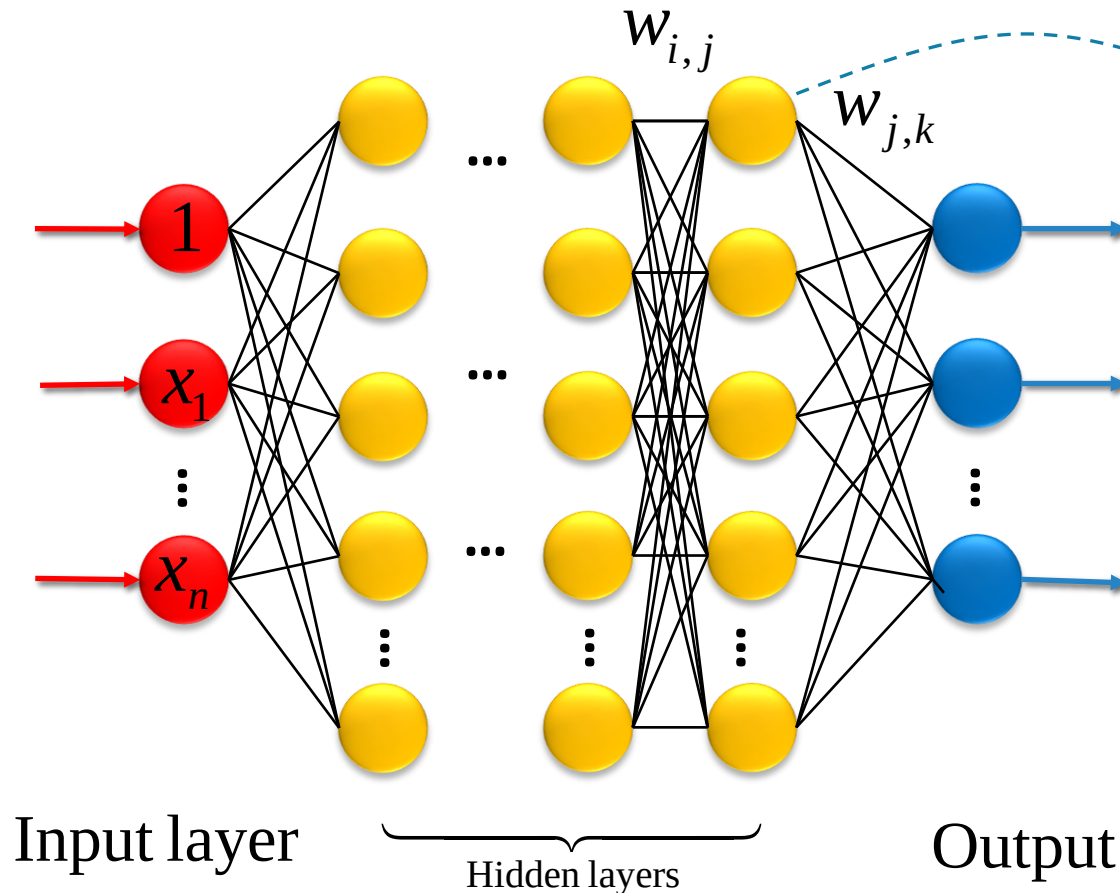
- Prediction (forward propagation): Predict the output based on weighting, input data and NN structure.

```
[Input] X : data input, y : label  
w ← 0  
A(1) = X; // Input layer  
  for l = 2:L //feedforward  
    Z(l) = W(l)A(l-1) + B(l);  
    A(l) = h(Z(l));  
  end  
Return A(L)
```

```
def layer(output_dim,input_dim,inputs, activation=None):  
    W = tf.Variable(tf.random_normal([input_dim, output_dim]))  
    b = tf.Variable(tf.random_normal([1, output_dim]))  
    XWb = tf.matmul(inputs, W) + b  
    if activation is None:  
        outputs = XWb  
    else:  
        outputs = activation(XWb)  
    return outputs
```

Multilayer Perceptron

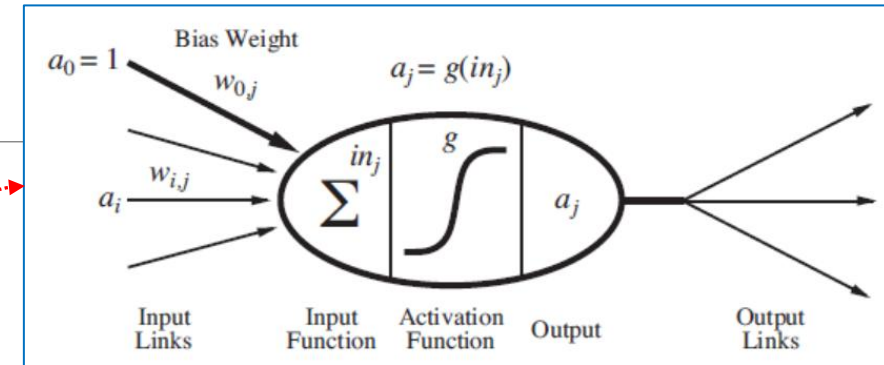
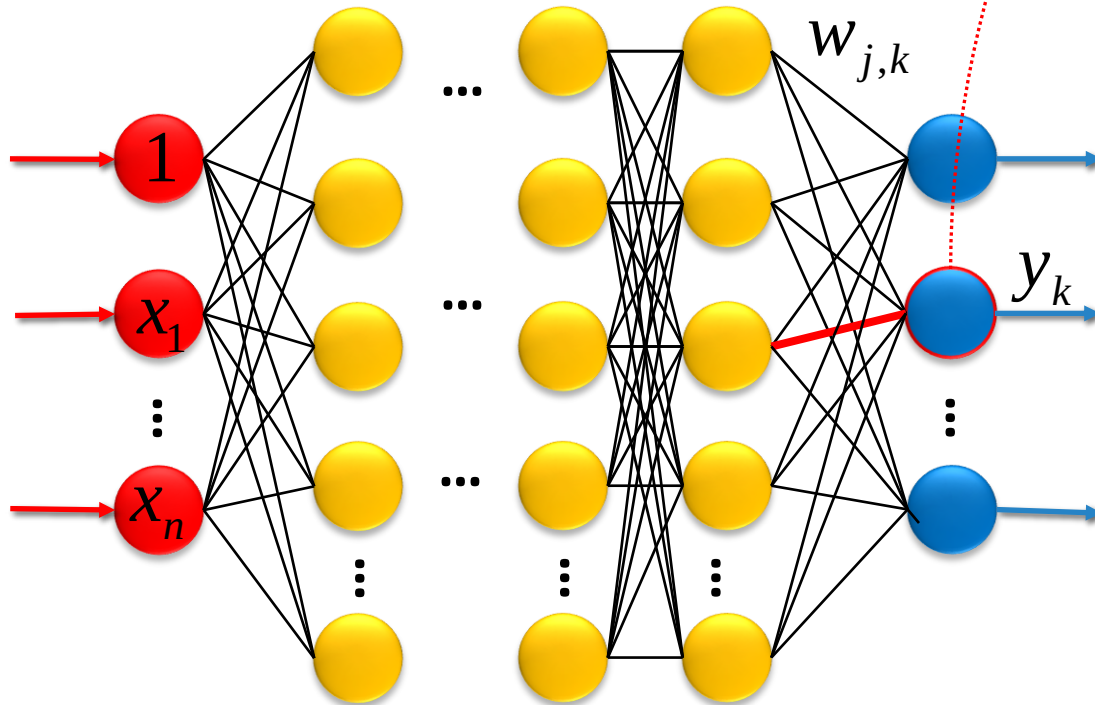
- Update (Backward): Update weighting based on input training data.



$w_{i,j}$: The weighting of i^{th} input node to j^{th} output node

Multilayer Perceptron

- j to k node weighting



$$Loss_k = (y_k - a_k)^2$$

y_k : labeled output
 a_k : NN output (prediction)

$$\frac{\partial Loss_k}{\partial w_{j,k}} = -2(y_k - a_k) \frac{\partial a_k}{\partial w_{j,k}} = -2(y_k - a_k) \frac{\partial g(in_k)}{\partial w_{j,k}}$$

$$= -2(y_k - a_k) g'(in_k) \frac{\partial in_k}{\partial w_{j,k}}$$

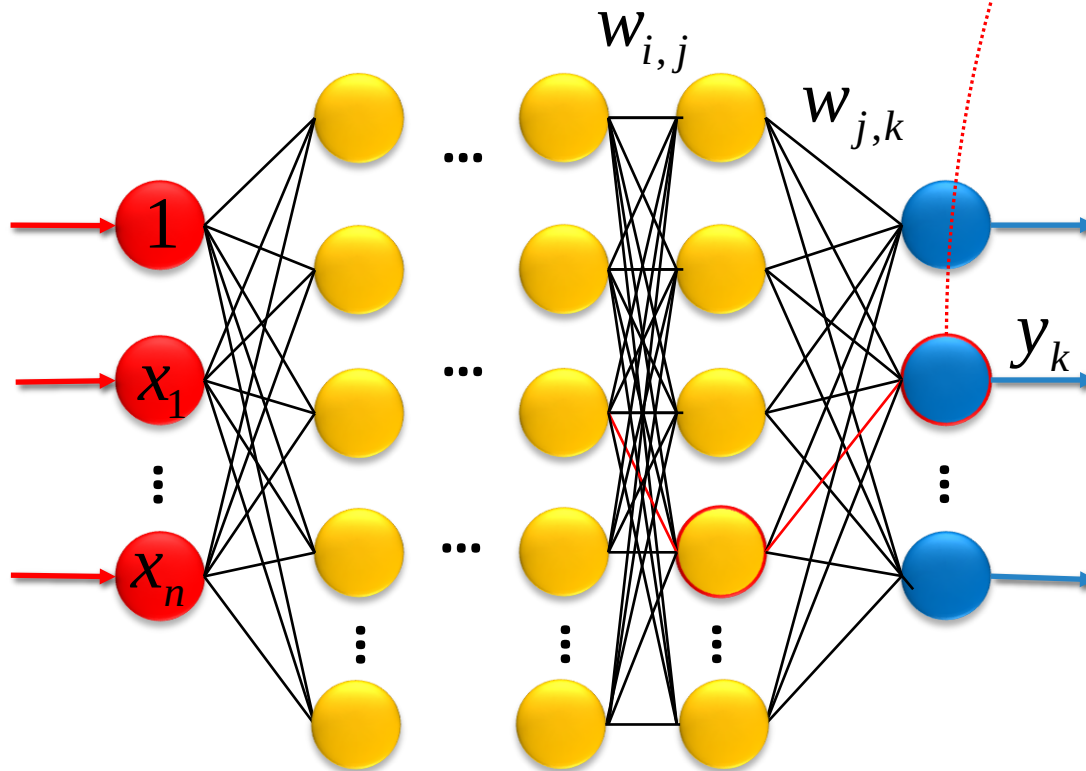
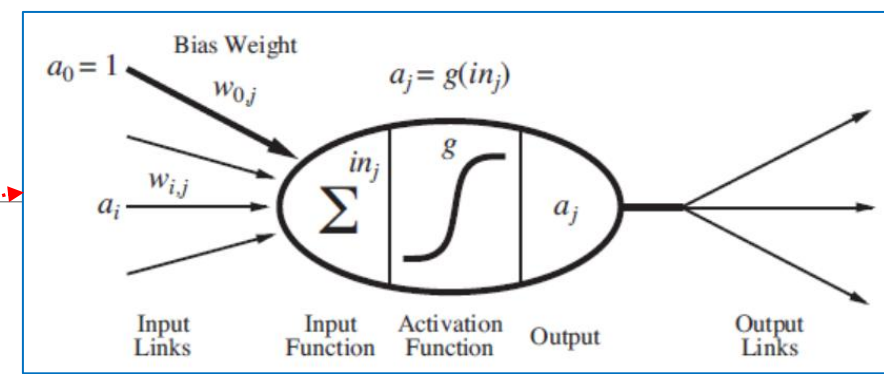
$$= -2(y_k - a_k) g'(in_k) \frac{\partial \left(\sum_j w_{j,k} a_j \right)}{\partial w_{j,k}}$$

$$= -2 \underbrace{(y_k - a_k) g'(in_k)}_{\Delta_k} a_j$$

$$w_{j,k} \leftarrow w_{j,k} + \alpha a_j \Delta_k$$

Multilayer Perceptron

- i to j node weighting



$$Loss_k = (y_k - a_k)^2$$

y_k : labeled output
 a_k : NN output (prediction)

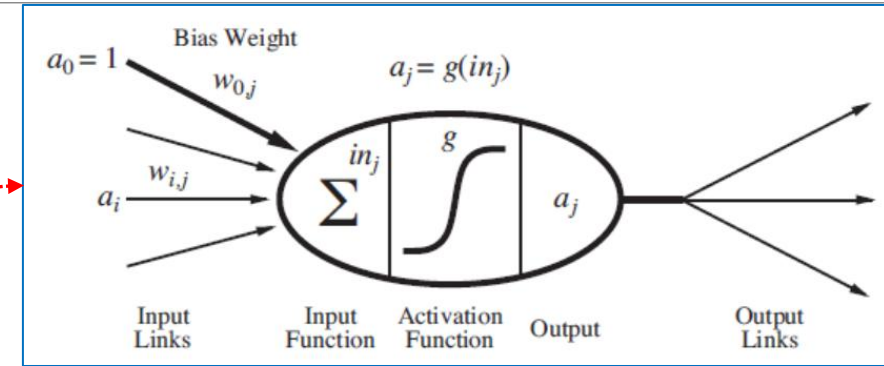
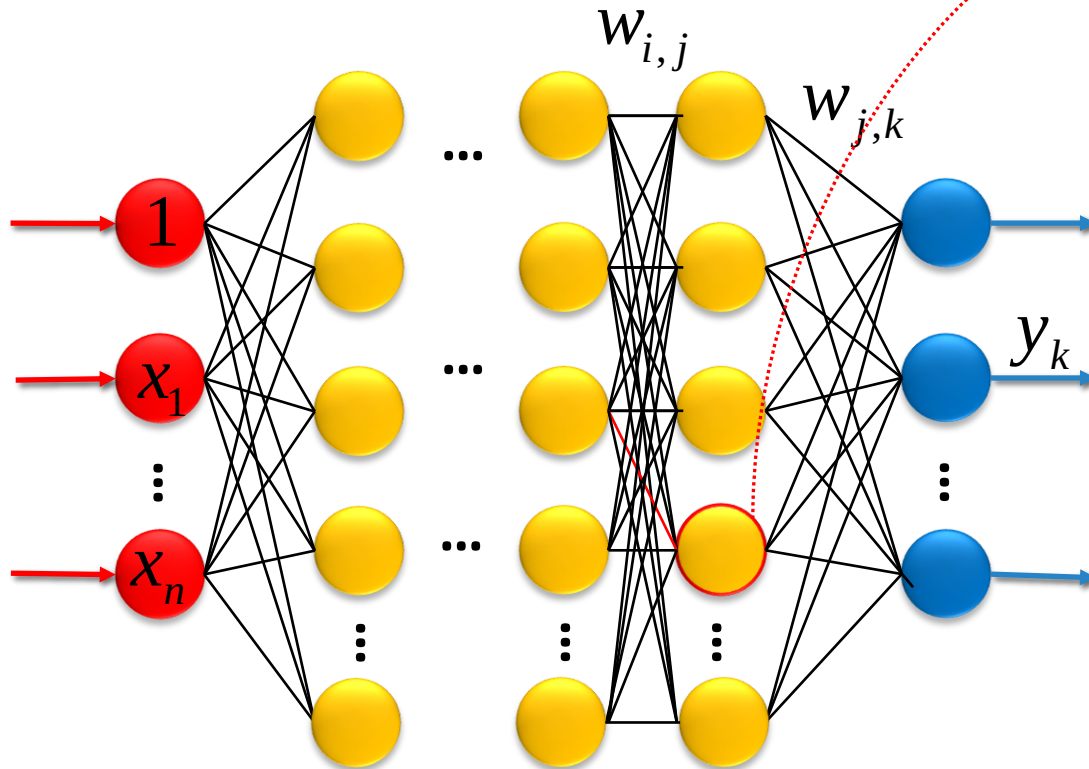
$$\frac{\partial Loss_k}{\partial w_{i,j}} = -2(y_k - a_k) \frac{\partial a_k}{\partial w_{i,j}} = -2(y_k - a_k) \frac{\partial g(in_k)}{\partial w_{i,j}}$$

$$= -2(y_k - a_k) g'(in_k) \frac{\partial g(in_k)}{\partial w_{i,j}} = -2\Delta_k \frac{\partial \left(\sum_j w_{j,k} a_j \right)}{\partial w_{i,j}}$$

$$= -2\Delta_k w_{j,k} \frac{\partial a_j}{\partial w_{i,j}} = -2\Delta_k w_{j,k} \frac{\partial g(in_j)}{\partial w_{i,j}}$$

Multilayer Perceptron

- i to j node weighting



$$\frac{\partial Loss_k}{\partial w_{i,j}} = -2\Delta_k w_{j,k} \frac{\partial g(in_j)}{\partial w_{i,j}}$$

$$= -2\Delta_k w_{j,k} g'(in_j) \frac{\partial in_j}{\partial w_{i,j}} = -2\Delta_k w_{j,k} g'(in_j) \frac{\partial \left(\sum_i w_{i,j} a_i \right)}{\partial w_{i,j}}$$

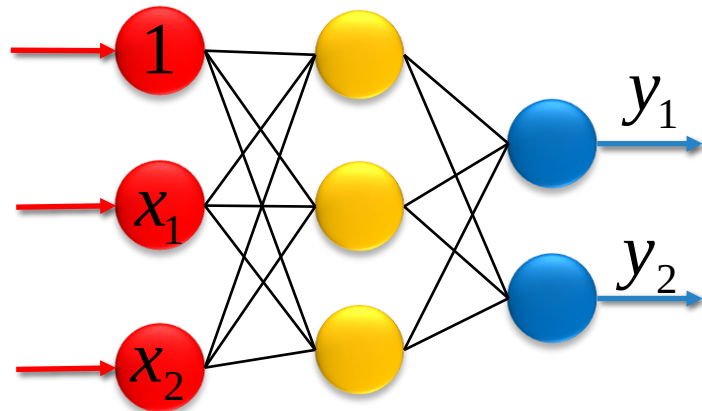
$$= -2\Delta_k w_{j,k} g'(in_j) a_i = -a_i \Delta_j$$

$$w_{i,j} \leftarrow w_{i,j} + \alpha a_i \Delta_j$$

Multilayer Perceptron

- Illustration of MLP (linear $\rightarrow$ logistic regression $\rightarrow$ MLP)

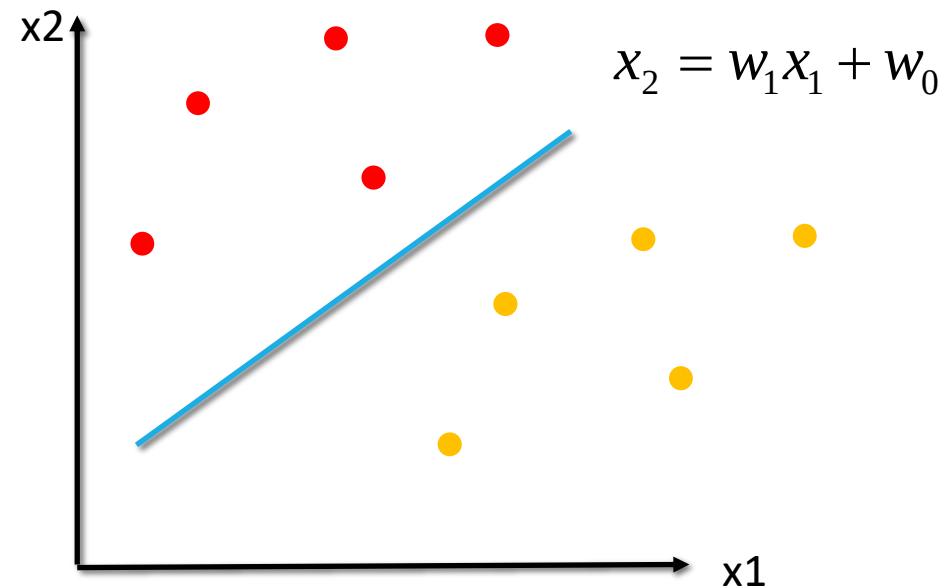
- Nonlinear functions
- Feature extraction



$$y = \mathbf{x}^T \mathbf{w}$$

$$w_2 x_2 + w_1 x_1 + w_0 = 0$$

$$x_2 + \frac{w_1}{w_2} x_1 + \frac{w_0}{w_2} > 0 \rightarrow \text{class1}$$



```

function BACK-PROP-LEARNING(examples, network) returns a neural network
  inputs: examples, a set of examples, each with input vector  $\mathbf{x}$  and output vector  $\mathbf{y}$ 
           network, a multilayer network with  $L$  layers, weights  $w_{i,j}$ , activation function  $g$ 
  local variables:  $\Delta$ , a vector of errors, indexed by network node

  for each weight  $w_{i,j}$  in network do
     $w_{i,j} \leftarrow$  a small random number
  repeat
    for each example  $(\mathbf{x}, \mathbf{y})$  in examples do
      /* Propagate the inputs forward to compute the outputs */
      for each node  $i$  in the input layer do
         $a_i \leftarrow x_i$ 
      for  $\ell = 2$  to  $L$  do
        for each node  $j$  in layer  $\ell$  do
           $in_j \leftarrow \sum_i w_{i,j} a_i$ 
           $a_j \leftarrow g(in_j)$ 
      /* Propagate deltas backward from output layer to input layer */
      for each node  $j$  in the output layer do
         $\Delta[j] \leftarrow g'(in_j) \times (y_j - a_j)$ 
      for  $\ell = L - 1$  to  $1$  do
        for each node  $i$  in layer  $\ell$  do
           $\Delta[i] \leftarrow g'(in_i) \sum_j w_{i,j} \Delta[j]$ 
      /* Update every weight in network using deltas */
      for each weight  $w_{i,j}$  in network do
         $w_{i,j} \leftarrow w_{i,j} + \alpha \times a_i \times \Delta[j]$ 
  until some stopping criterion is satisfied
  return network

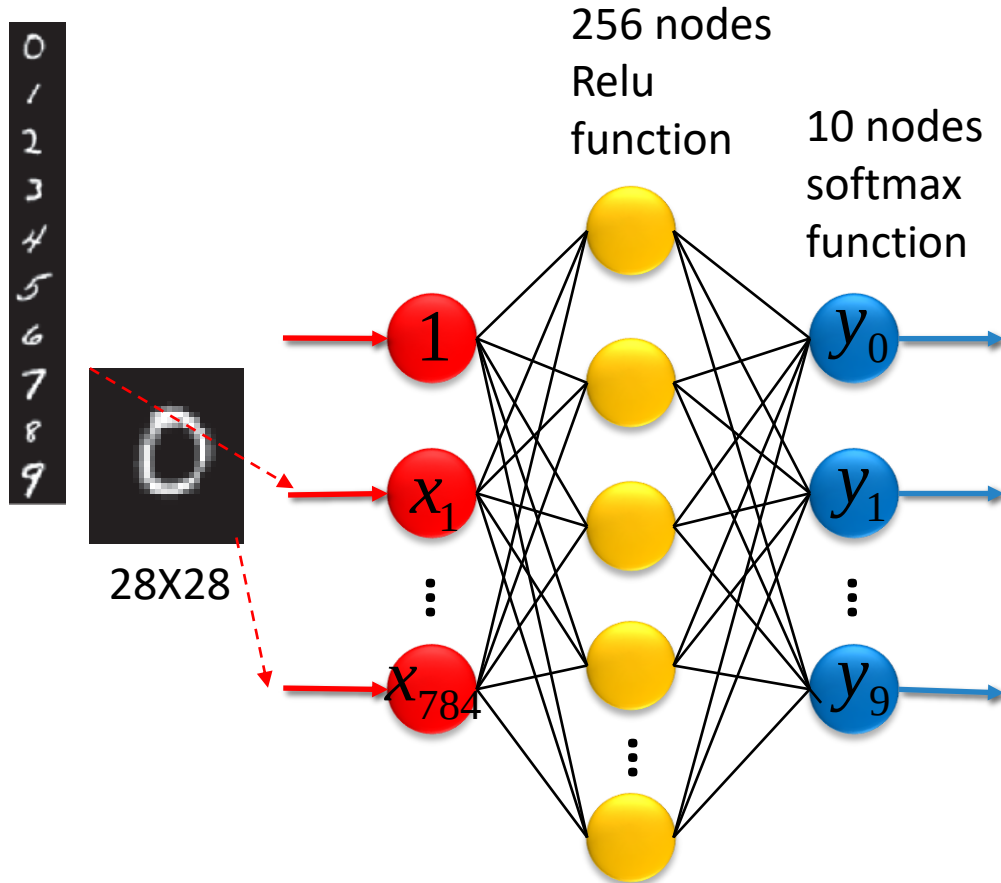
```

It's easy to implement it via Tensorflow!

MLP for MNIST

- MLP for MINST

$$785 * 256 + 256 * 10 \\ = 203,520$$



According to $y_0 \sim y_9$, MLP predicts the output.

The final accuracy is 94%

**However, it doesn't make sense to take each pixel and each weight of MLP for image classification
It takes too much memory and computation!**

We should use meaningful image features instead of raw data.

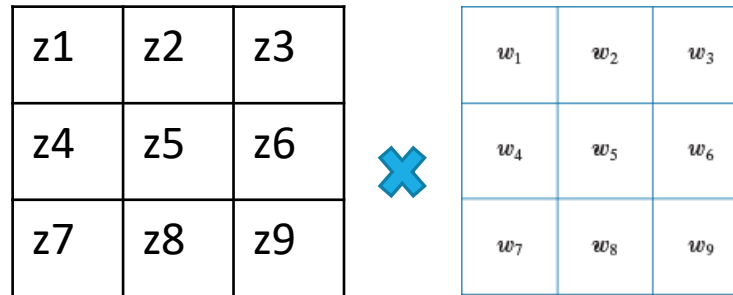
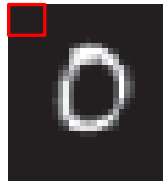
<http://yann.lecun.com/exdb/mnist/>

Outline

- Neural Network
- Multilayer Perceptron
- **Weighted Filter**
- Convolution Neural Network
- EX: MNIST
- Deep Reinforcement Learning
- EX: Atari with DQN
- EX: Turtlebot3 with DQN

Weighted Filter

- In image processing, researchers adopt some filters to extract features (e.g., edge), which are useful for detection.



$$y = \sum_i z_i w_i$$

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

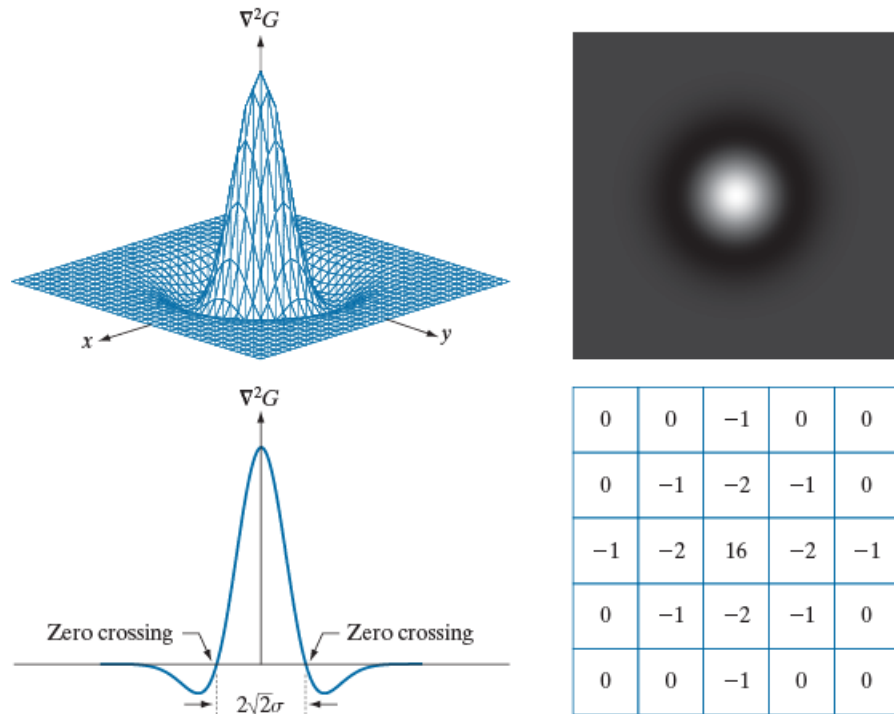
Sobel

Sobel is one kind of filters for edge detection.

[5] Rafael C. Gonzalez and Richard E. Woods, "Digital Image Processing," Pearson, 4th edition, 2018.

Weighted Filter

- Gaussian filter is also popular for edge detection. It has parameters for different effects.



a b
c d

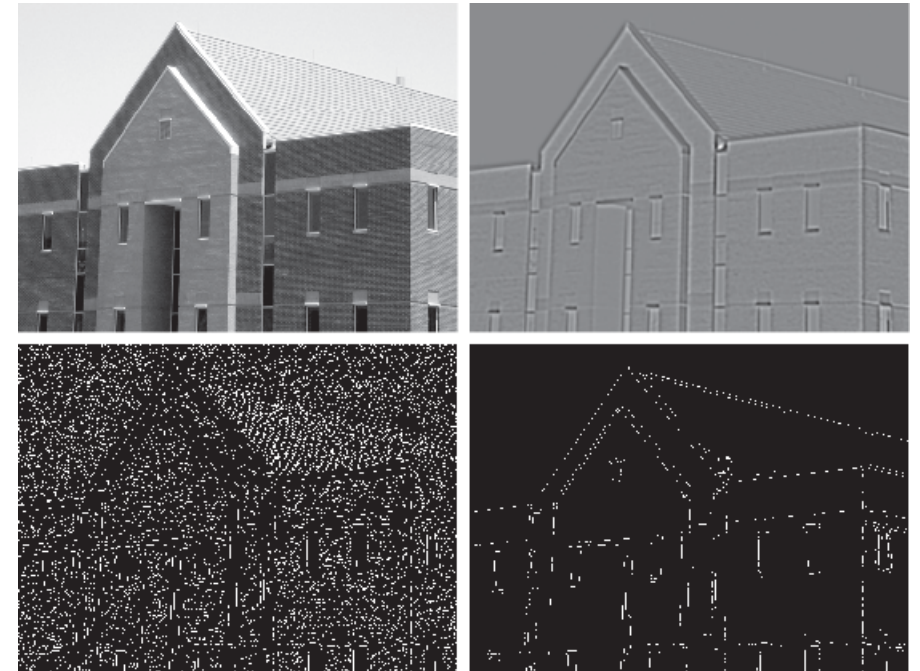
FIGURE 10.22

(a) Image of size 834×1114 pixels, with intensity values scaled to the range $[0, 1]$.

(b) Result of Steps 1 and 2 of the Marr-Hildreth algorithm using $\sigma = 4$ and $n = 25$.

(c) Zero crossings of (b) using a threshold of 0 (note the closed-loop edges).

(d) Zero crossings found using a threshold equal to 4% of the maximum value of the image in (b). Note the thin edges.



How about learning the weighing of filters from training data via neural networks?

Outline

- Neural Network
- Multilayer Perceptron
- Weighted Filter
- **Convolution Neural Network**
- EX: MNIST
- Deep Reinforcement Learning
- EX: Atari with DQN
- EX: Turtlebot3 with DQN

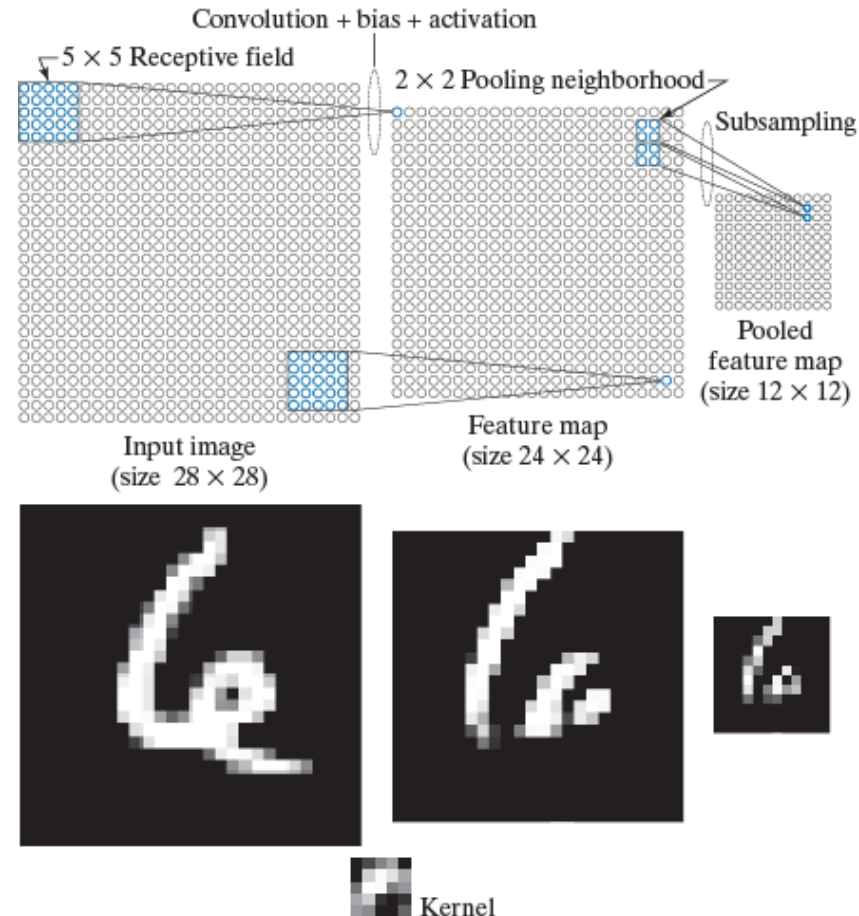
Convolution Neural Network

- Convolution Neural Network (CNN) was proposed by Prof. Yann LeCun. The major concept is to learn the weighting of filters from data.

FIGURE 12.41

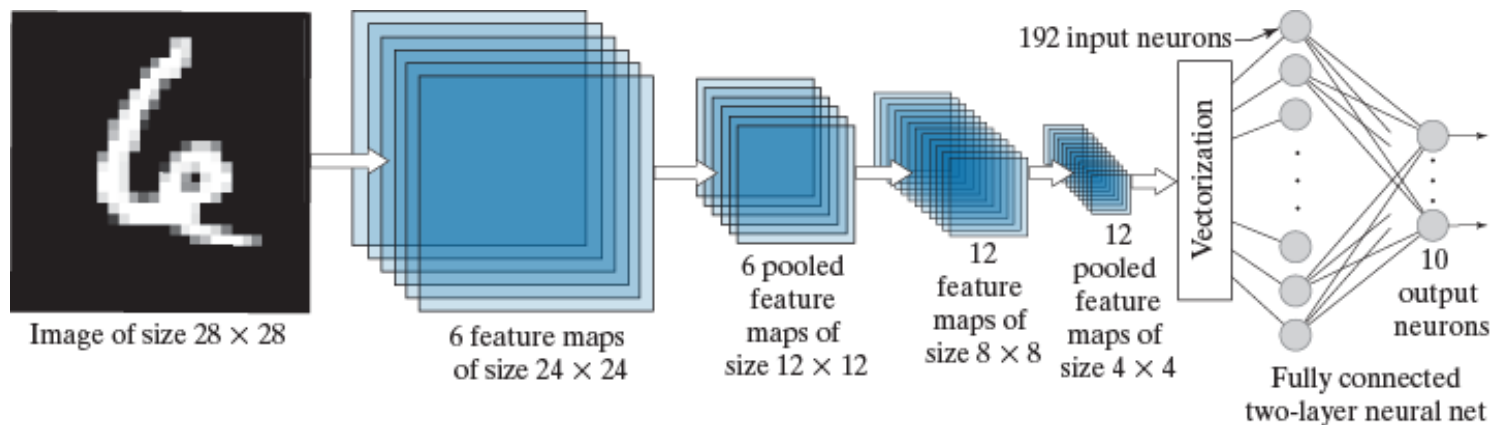
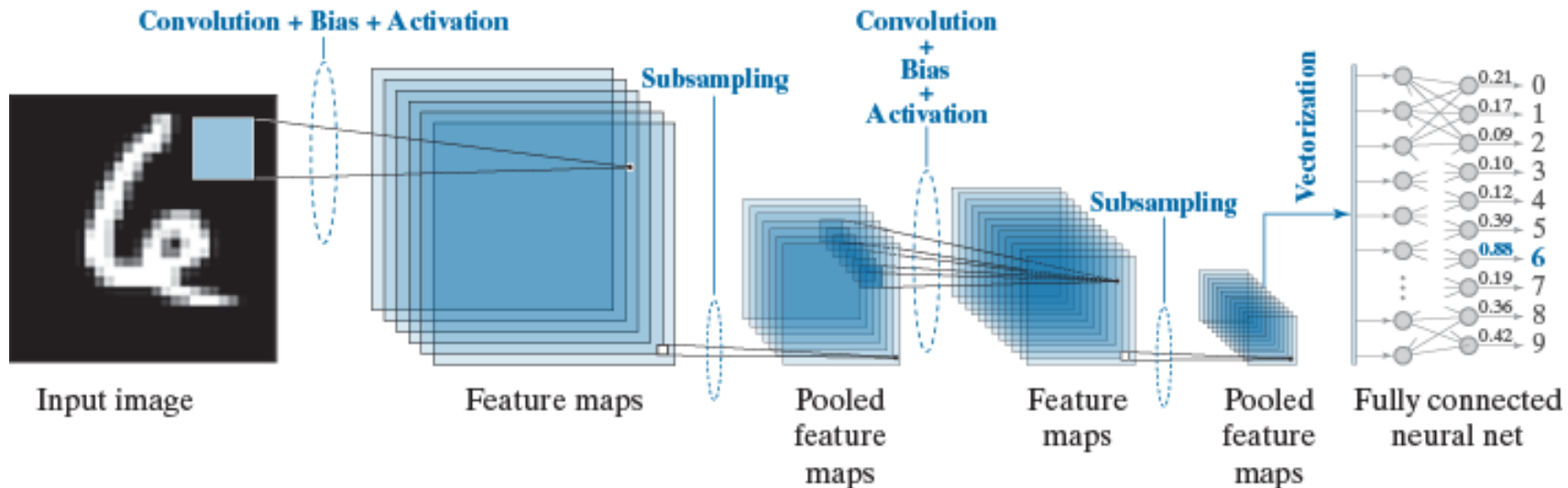
Top row: How the sizes of receptive fields and pooling neighborhoods affect the sizes of feature maps and pooled feature maps.

Bottom row: An image example. This figure is explained in more detail in Example 12.17. (Image courtesy of NIST.)



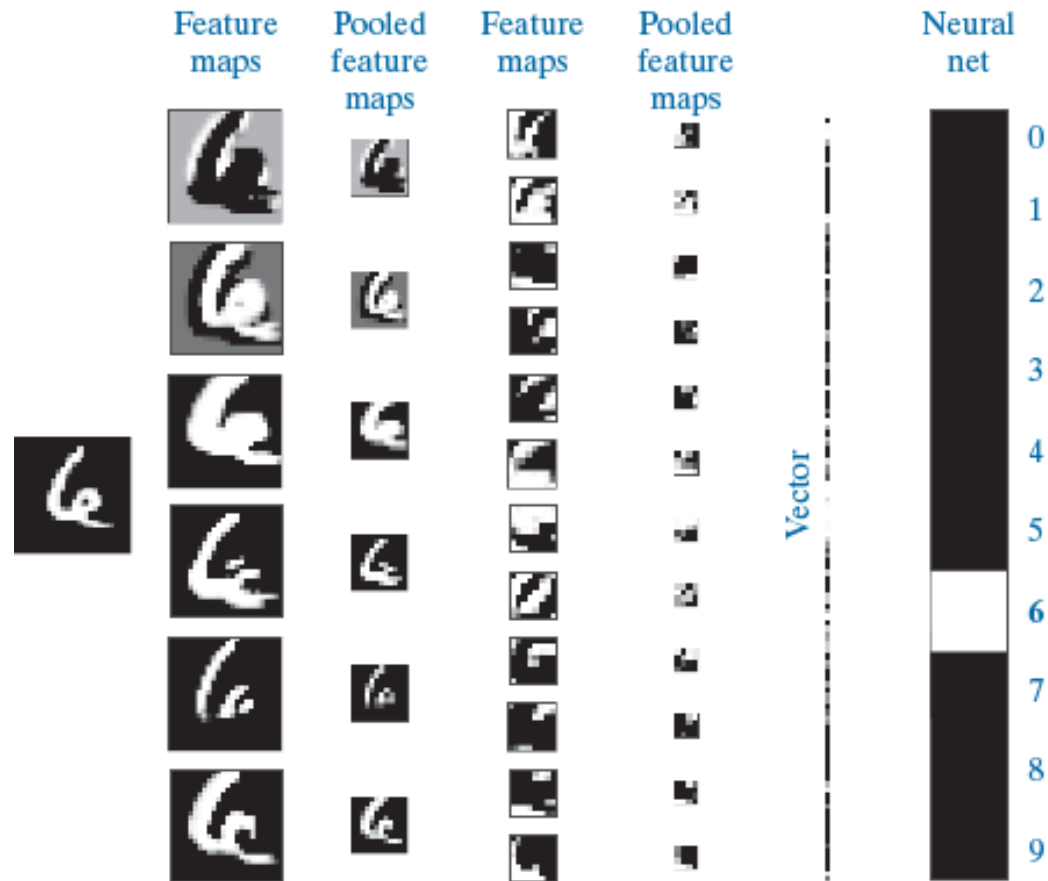
Convolution Neural Network

- CNN for MNIST



Convolution Neural Network

- The images for each layers



The final accuracy is ~99%

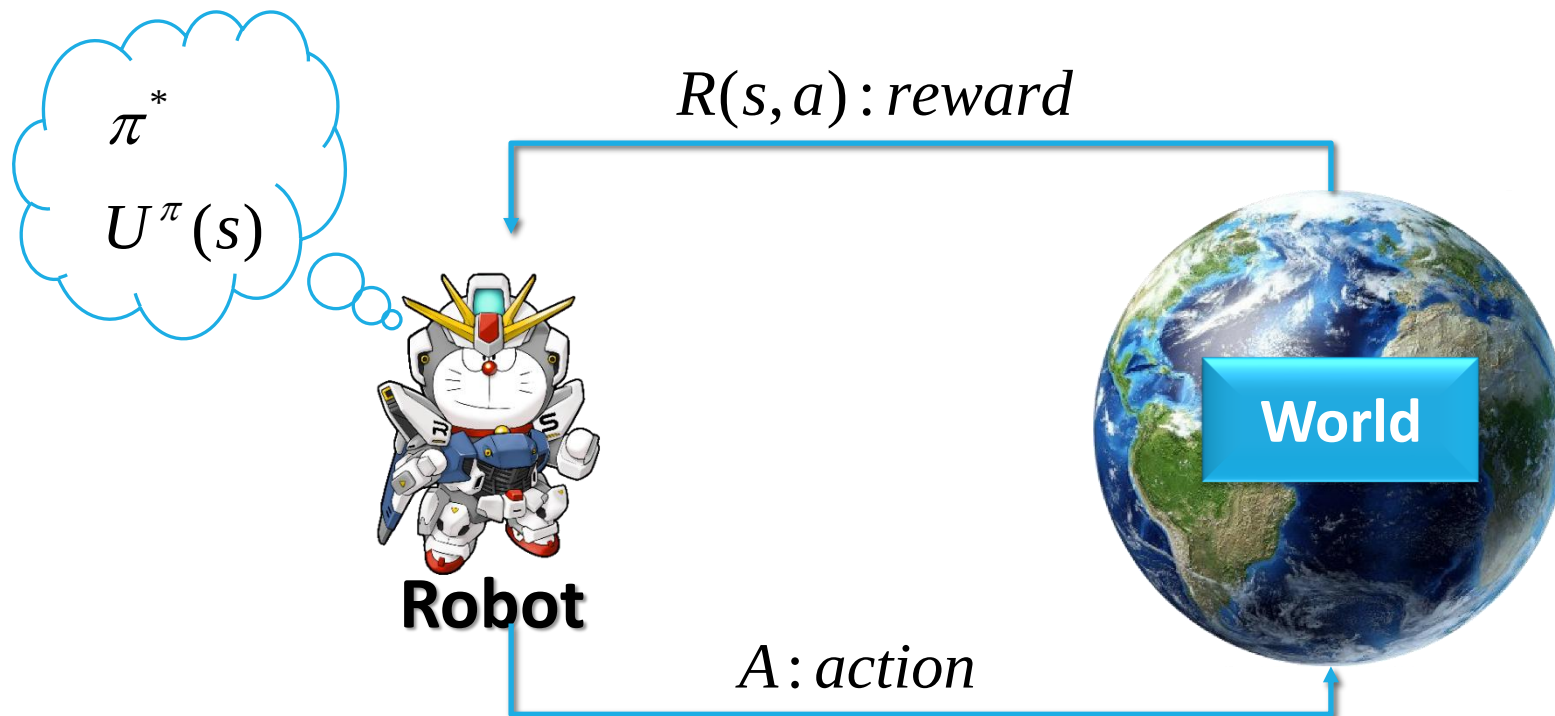
However, it still takes a lot of computation.
Hence, CNN usually needs GPUs to speed up the training stages (backpropagation).

Outline

- Neural Network
- Multilayer Perceptron
- Weighted Filter
- Convolution Neural Network
- EX: MNIST
- **Deep Reinforcement Learning**
- EX: Atari with DQN
- EX: Turtlebot3 with DQN

Reinforcement learning

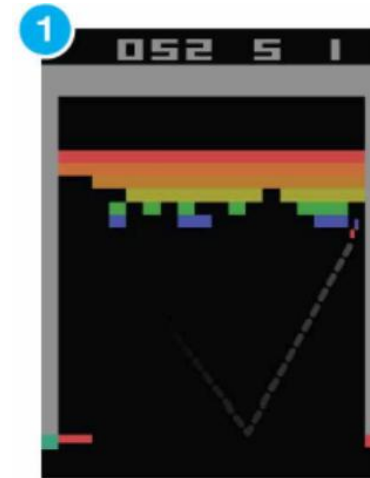
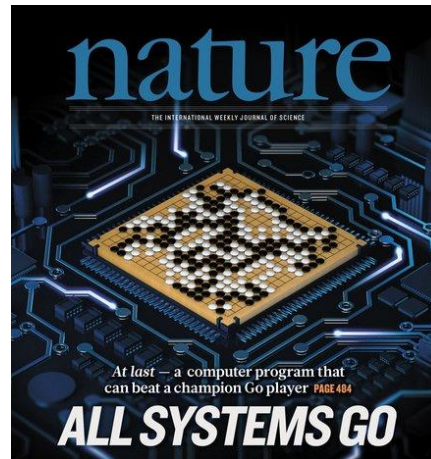
- Let the robot interact with the world and receive the reward immediately. RL is inspired by animals. Animals learn the world via interactions!



3				+1
2				-1
1	START			
	1	2	3	4

Deep Reinforcement Learning

- During the past 10 years, robots are able to do amazing **control** tasks (~60 DOFs) using RL.
- In 2015, AlphaGo can beat the best human players.
- In 2015, AI can play 49 Atari games.
- RL+DL is the future of AI?



Deep Reinforcement Learning

- Since the successes in deep learning of image classification, researcher in RL adopted NN to learn Q functions, so called deep Q-network (DQN).
- In Q-learning, the robot tries to learn Q-functions. Based on Q-functions, the robot can find the optimal decisions.
- We mentioned two approaches for learning Q-functions.
 - Tubular Q-learning
 - Function based Q-learning
- DQN is to learn Q-function via MLP (nonparametric approximation).

Deep Reinforcement Learning

- Why DRL is the future of AI?
 1. DL is a nonlinear function approximation
 2. DL is a nonparametric approximation (no hand crafting any more!)
 3. DL can deal with raw data input (e.g., images or voices)
 4. DL can output any functions (e.g., +1/-1 or continuous values)
 5. DL is an end-to-end solution for RL

Deep Reinforcement Learning

- Q-learning
- The formulation of Q-learning is the same. The key is to adopt NN for function approximation

$$\hat{Q}(s, a) = \sum_i w_i f_i(s, a)$$

w : weighting vector

f : features

Linear regression



$$\hat{Q}(s, a, W) = h(WX)$$

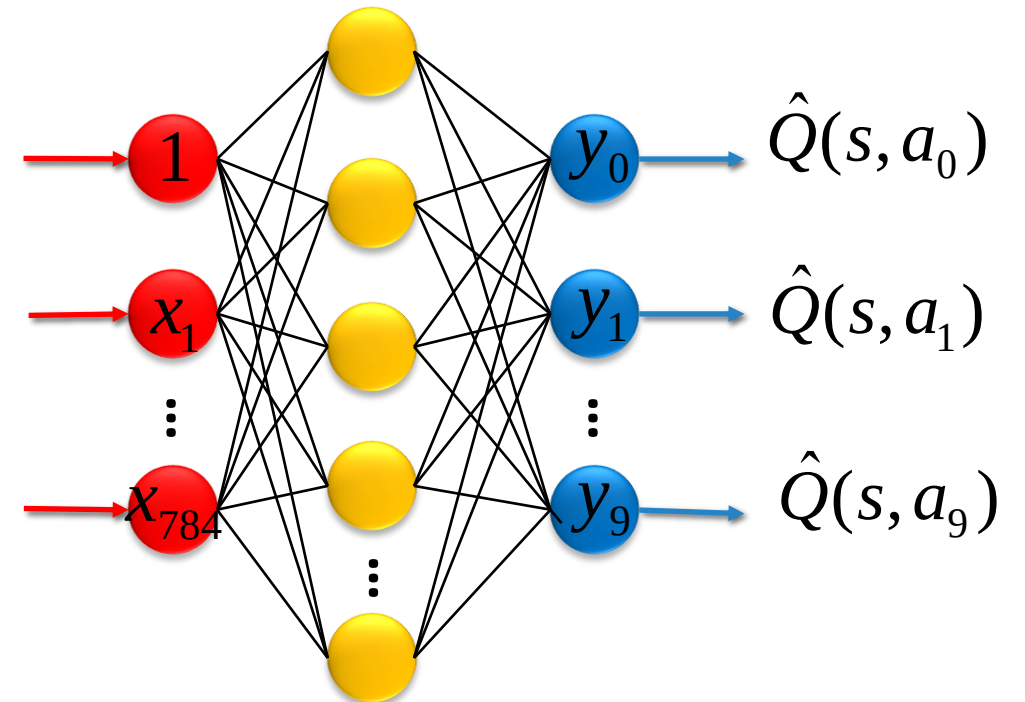
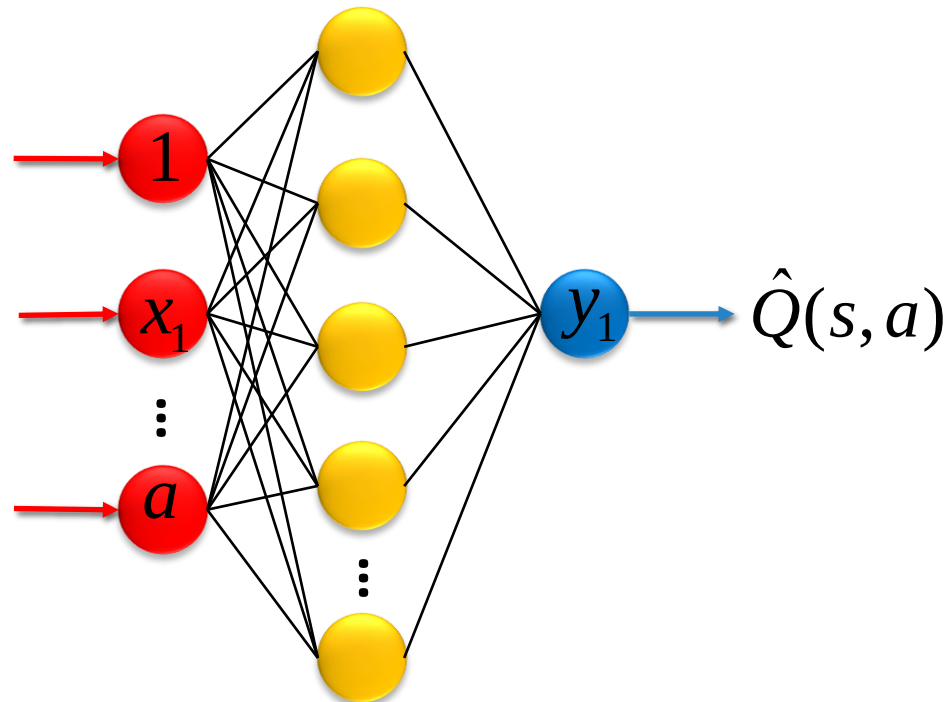
W : weighting vector

h : activation function

Nonlinear regression

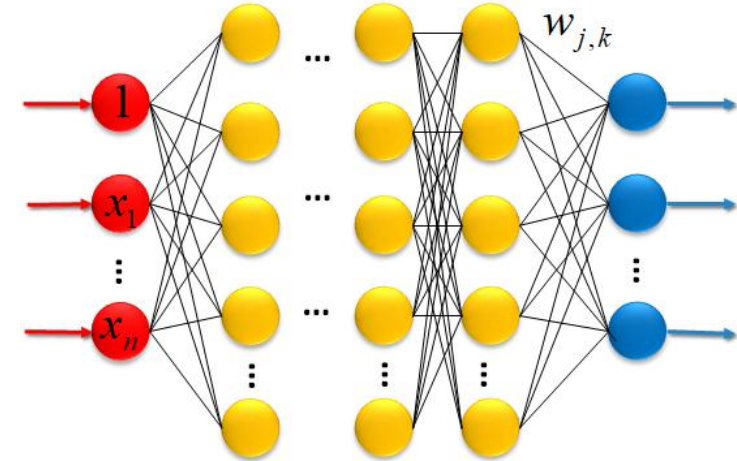
Deep Reinforcement Learning

- Q-learning
- Which NN structure is better for Q-learning?



Deep Reinforcement Learning

- What you need to do?
 - Define the state and action
 - Assign reward
 - Design a NN structure



[*GIVEN*]

S : state

A : action

$R(s, a)$: reward

γ : discount

α : learning rate

[*Find*]

$Q(s, a)$: action – utility function (Q function)

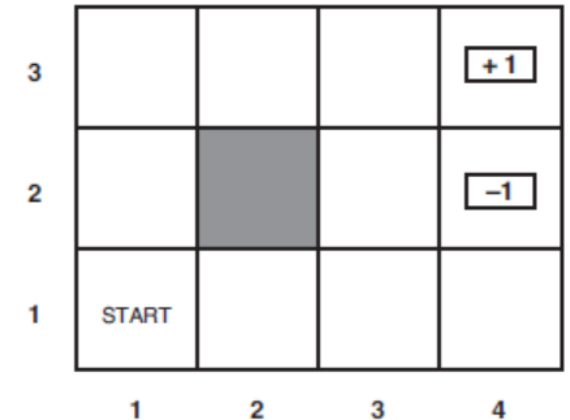
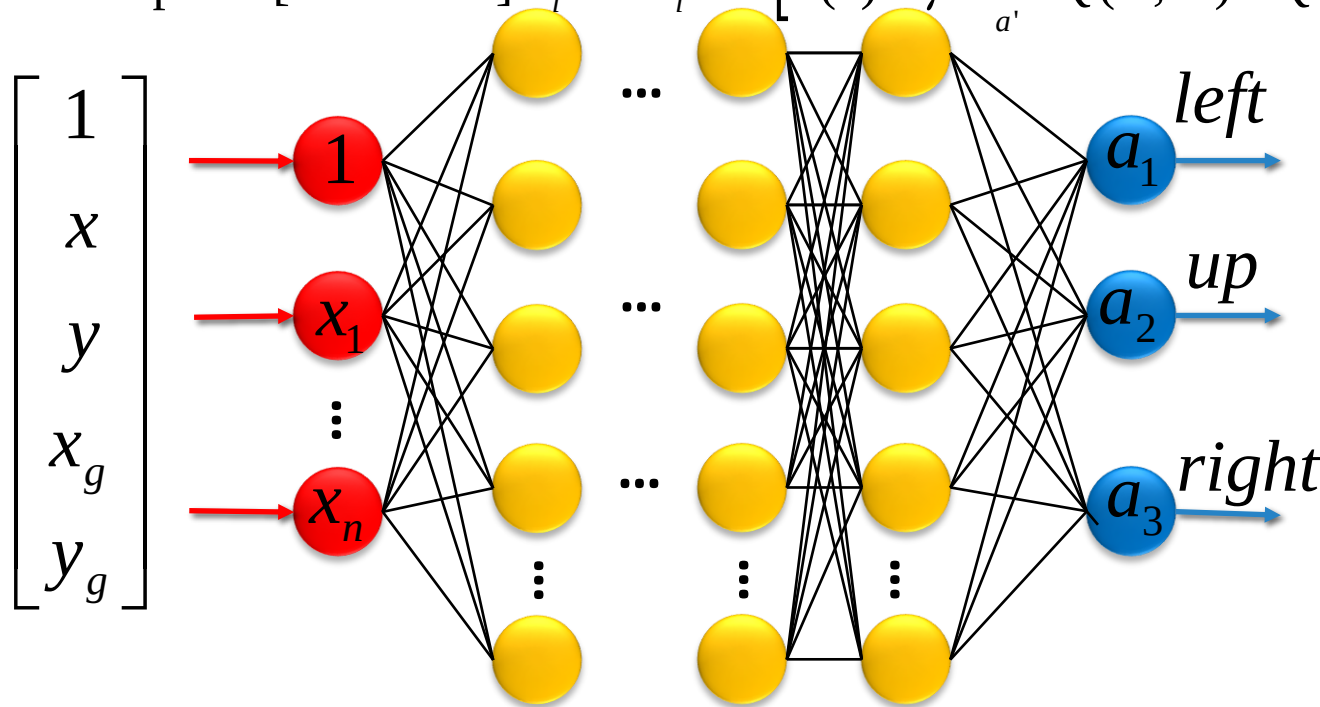
Deep Reinforcement Learning

- What you need to do?

1. run action based on $\hat{Q}(s,a,W)$ [forward] or exploration

and then get (s_i, a_i, s'_i, r_i)

2. update [backward] $w_i \leftarrow w_i + \alpha \left[R(s) + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s, a) \right] \frac{\partial \hat{Q}(s, a)}{\partial w_i}$



However, this approach is not stable!

1. Correlations between samples
2. The estimated Q is changing

To solve these issues, DeepMind proposed “experience replay.”

Deep Reinforcement Learning

- What you need to do?

1. update MLP parameters: $W' \leftarrow W$

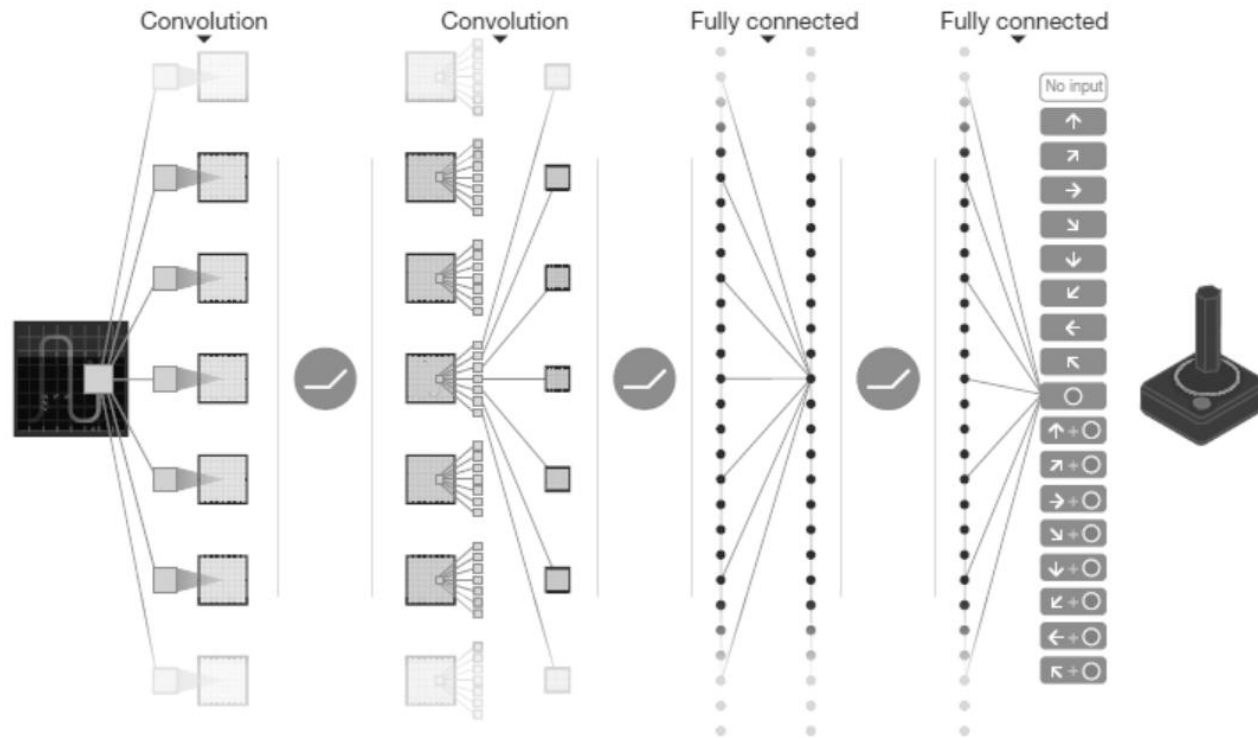
2. run N actions based on $\hat{Q}(s, a, W)$ [forward] or exploration and then get (s_i, a_i, s'_i, r_i) . Add these data to replay buffer(B).

3. sample K (s_j, a_j, s'_j, r_j) from B uniformly.

4. update [backward] $w_i \leftarrow w_i + \alpha \left[\underbrace{R(s) + \gamma \max_{a'} \hat{Q}(s', a', W') - \hat{Q}(s, a, W)} \right] \frac{\partial \hat{Q}(s, a, W)}{\partial w_i}$

DQN for Atari

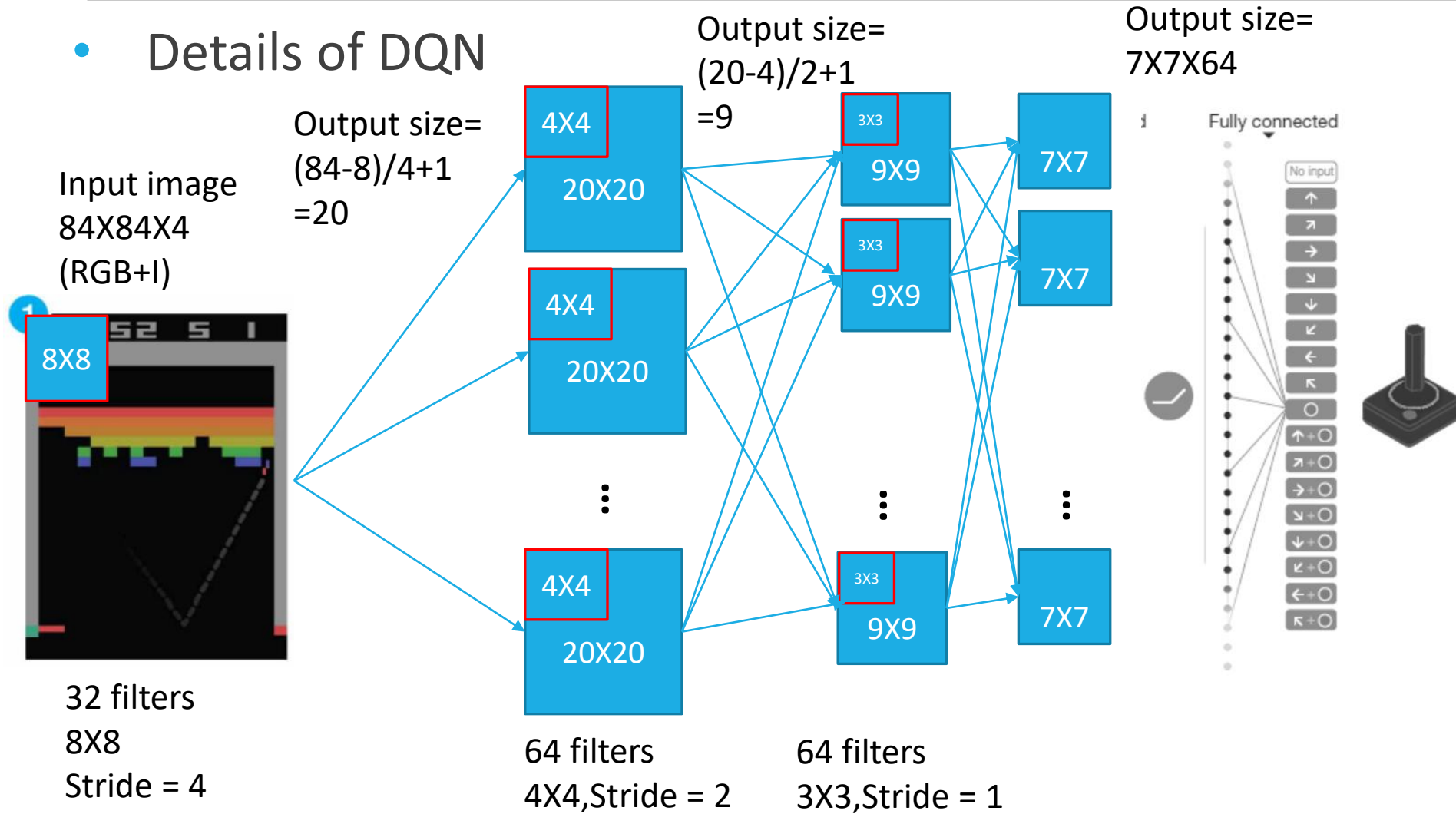
- DeepMind proposed a DQN for playing 48 kinds of Atari games. The robot's performance is better than human players in 29 games!



Mnih et al., “Human-level control through deep reinforcement learning,” Nature, 2015.

DQN for Atari

- Details of DQN

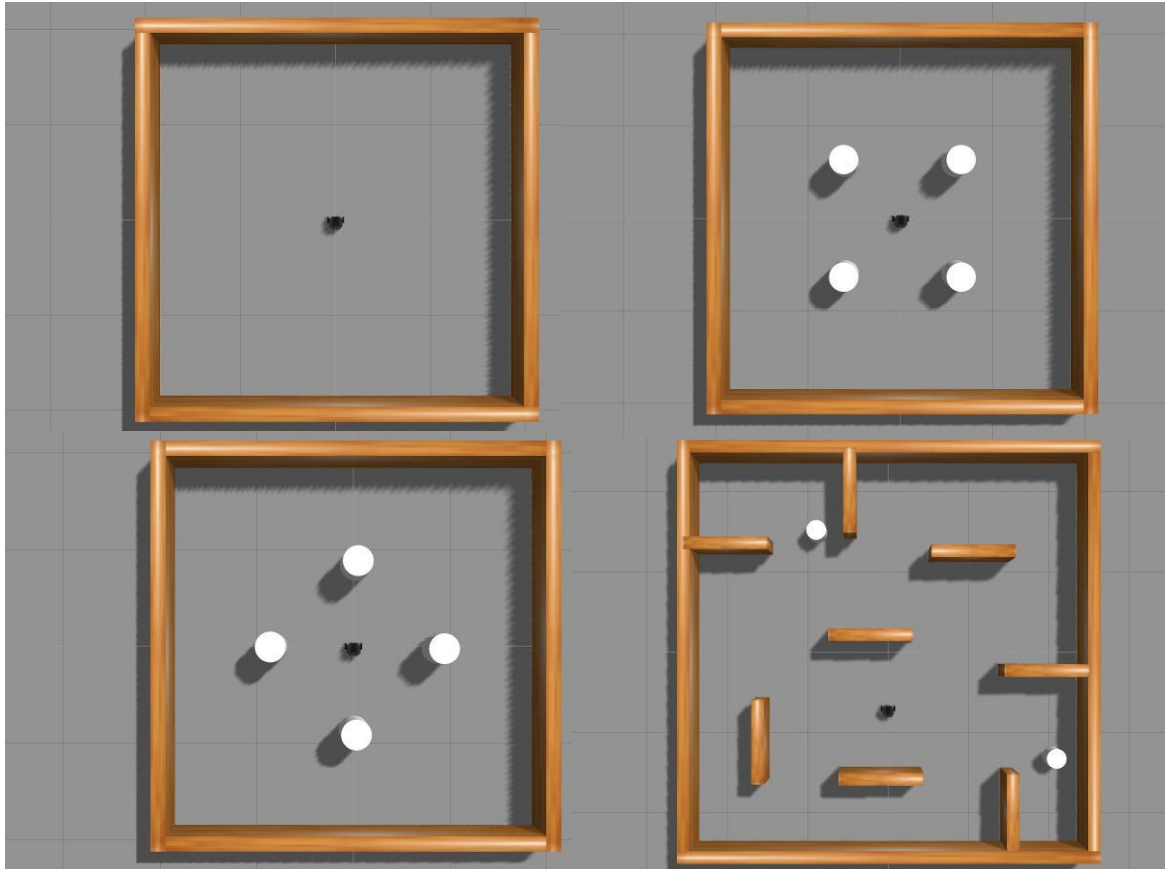


[Video](#)

<https://www.slideshare.net/onghaoyi/distributed-deep-qlearning>

Turtlebot3 with DQN

- Goal: Learn to arrive destination without hitting anything.



Example from https://github.com/ROBOTIS-GIT/turtlebot3_machine_learning.git

Turtlebot3 with DQN

- Goal: Learn to arrive destination without hitting anything.
- Sensors Input: The robot has a laser distance sensor (LDS) and encoder, which gets robot position.
- Actuator output: The robot can move via rotating two wheels.
- Let's define the MDP and then design the MLP.

[*GIVEN*]

S : *state*

A : *action*

$R(s, a)$: *reward*

γ : *discount*

α : *learning rate*

[*Find*]

$Q(s, a)$: *action – utility function (Q function)*

Turtlebot3 with DQN

- Let's define the MDP and then design the MLP.

[GIVEN]

S : state

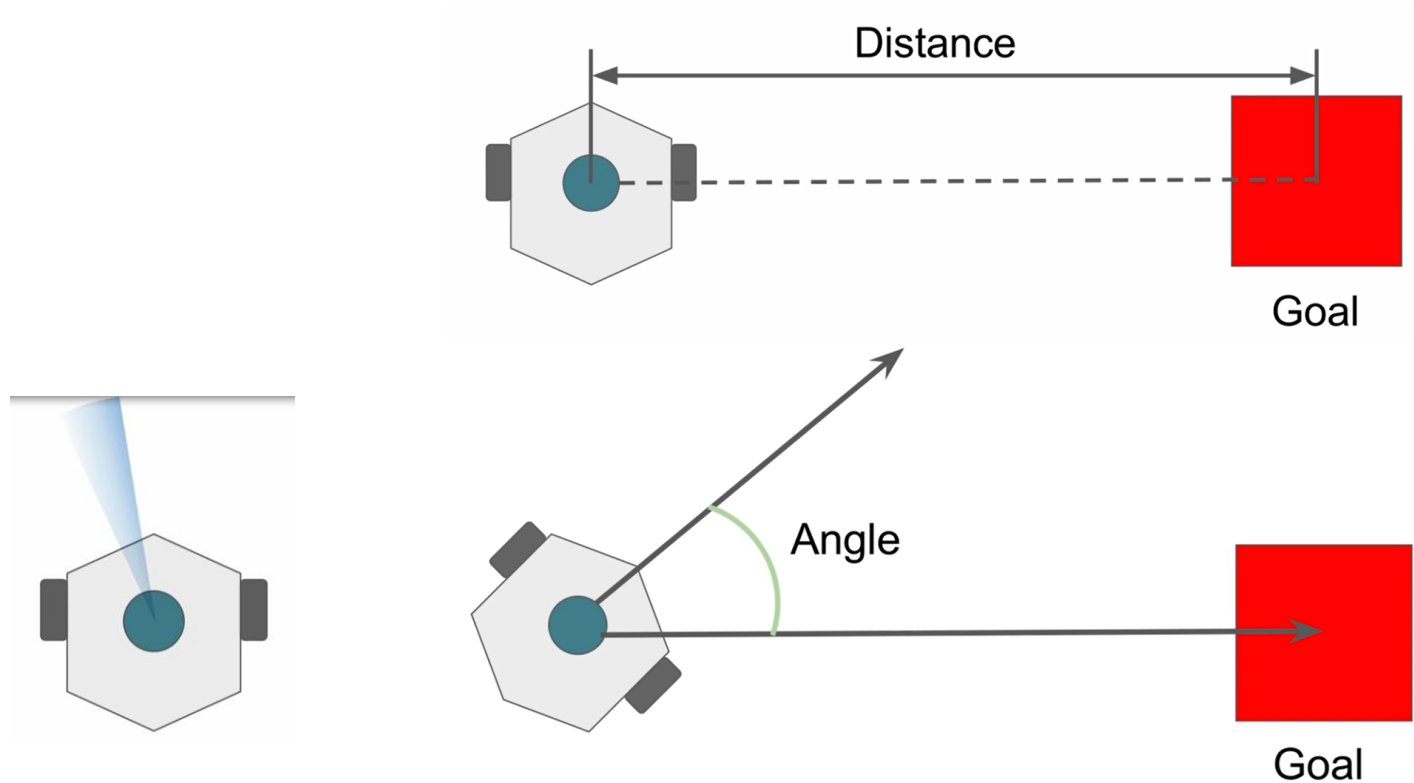
A : action

$R(s, a)$: reward

γ : discount

α : learning rate

$$S = \begin{bmatrix} \text{LDS}_{(24)} \\ \text{Distance} \\ \text{Angle} \end{bmatrix}$$



Turtlebot3 with DQN

- Let's define the MDP and then design the MLP.

[GIVEN]

S : state

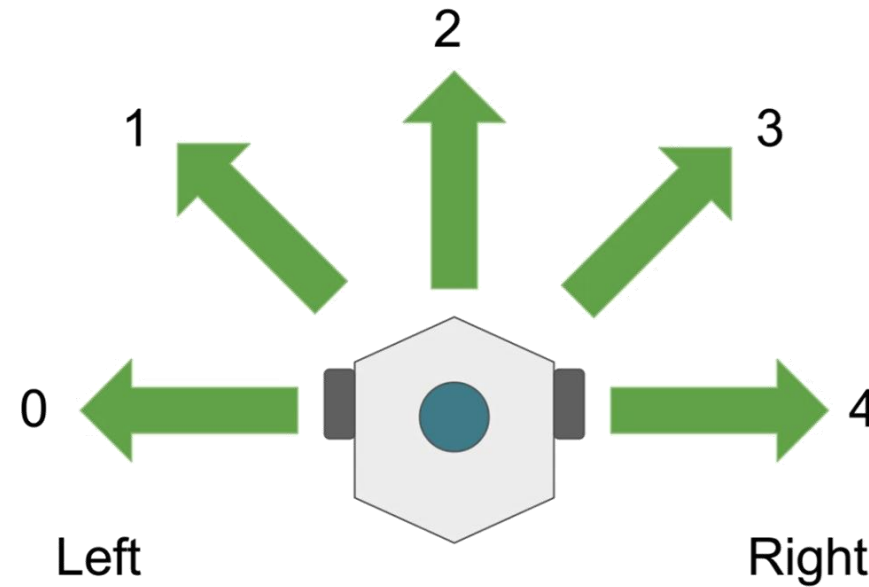
A : action

$R(s, a)$: reward

γ : discount

α : learning rate

$A = \{0, 1, 2, 3, 4\}$



0.15 m/s of linear velocity

Action	Angular velocity(rad/s)
0	-1.5
1	-0.75
2	0
3	0.75
4	1.5

Turtlebot3 with DQN

- Let's define the MDP and then design the MLP.

[GIVEN]

S : state

A : action

$R(s, a)$: reward

γ : discount

α : learning rate

$$\text{if } -\frac{1}{2}\pi < \theta < \frac{1}{2}\pi ,$$

$$R_\theta \geq 0$$

else

$$R_\theta < 0$$

$$\text{if } D_c < D_g ,$$

$$R_d > 2$$

else

$$1 < R_d \leq 2$$

$$R = R_\theta \cdot R_d$$

θ : Angle from TurtleBot3 to Goal

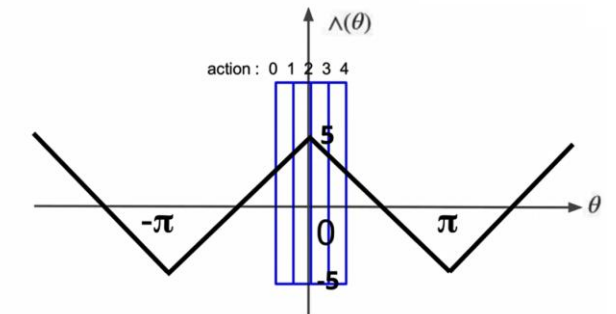
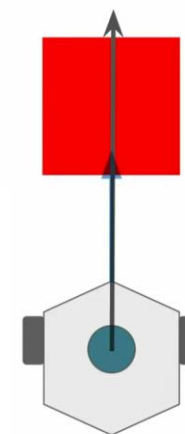
D_c : Current distance from Goal

D_g : Absolute distance from Goal

R_θ : Reward from θ

R_d : Reward from distance

R : Reward



$$\theta = \frac{\pi}{2} + \text{action} \cdot \frac{\pi}{8} + \phi$$

$$R_\theta = 5 \cdot (1 - \Lambda(\theta))$$

ϕ : Yaw of Turtlebot3

θ : Angle from goal

R_θ : Reward from angle

Turtlebot3 with DQN

- Let's define the MDP and then design the MLP.

[GIVEN]

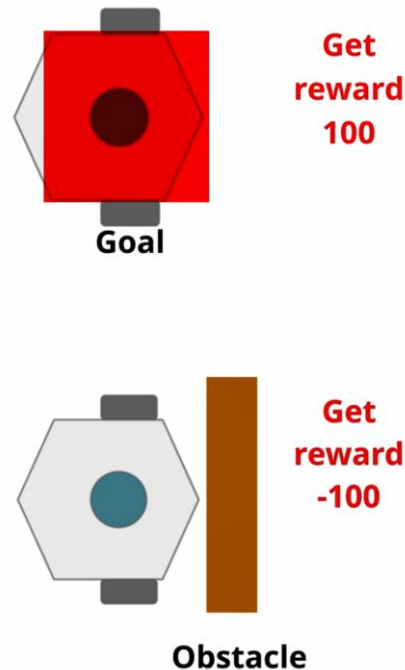
S : state

A : action

$R(s, a)$: reward

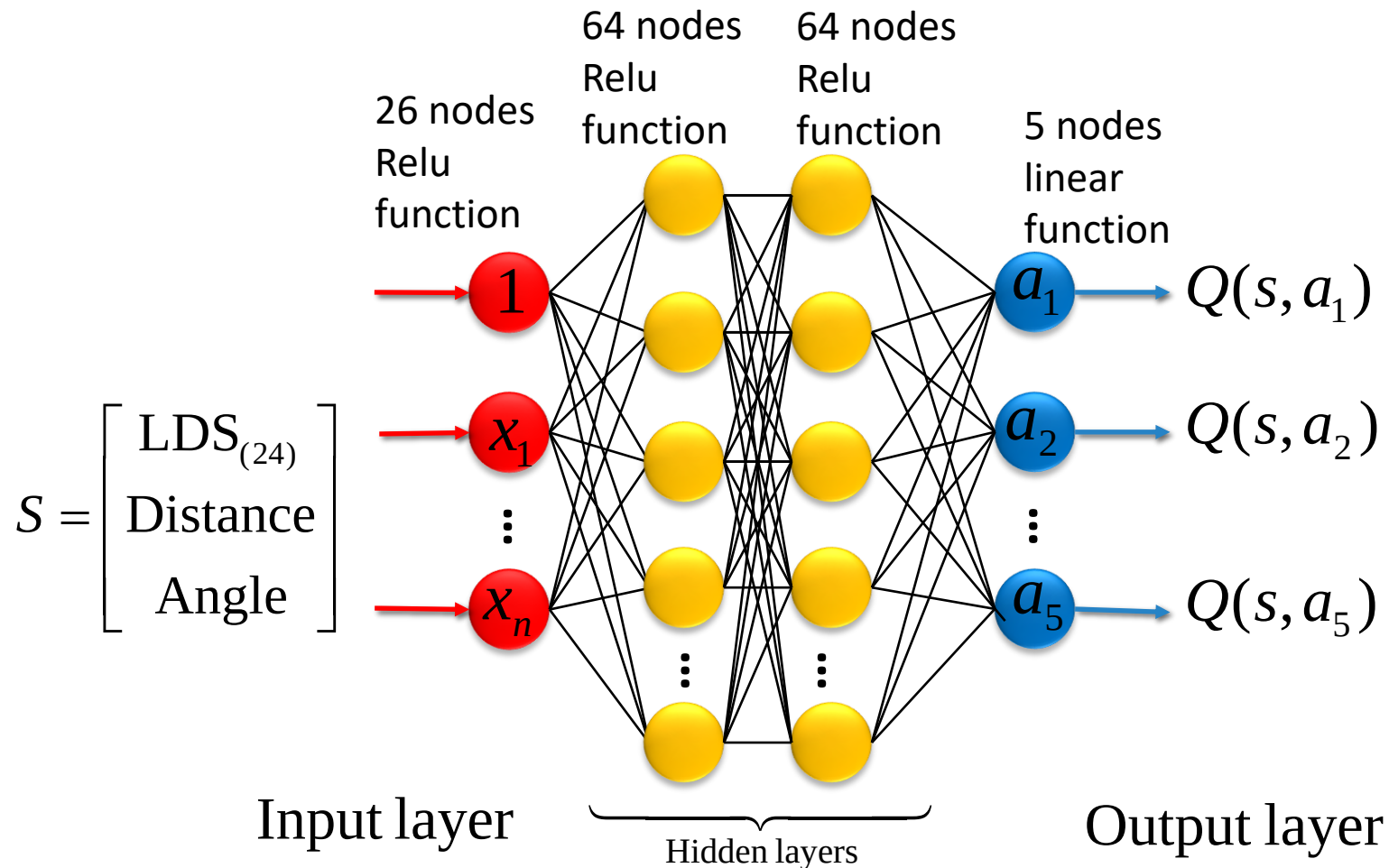
γ : discount

α : learning rate



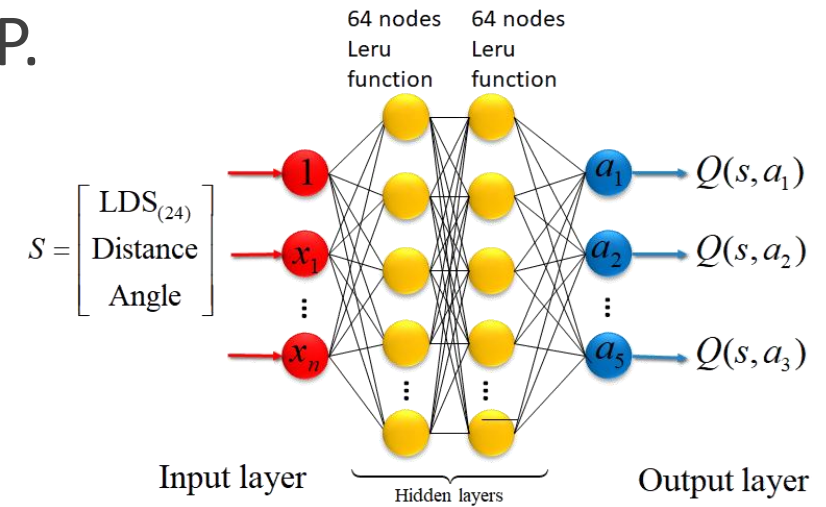
Turtlebot3 with DQN

- Let's define the MDP and then design the MLP.



Turtlebot3 with DQN

- Let's define the MDP and then design the MLP.



```
model.add(Dense(64, input_shape=(self.state_size,), activation='relu', kernel_initializer='lecun_uniform'))
model.add(Dense(64, activation='relu', kernel_initializer='lecun_uniform'))
model.add(Dropout(dropout))
model.add(Dense(self.action_size, kernel_initializer='lecun_uniform'))
model.add(Activation('linear'))
model.compile(loss='mse', optimizer=RMSprop(lr=self.learning_rate, rho=0.9, epsilon=1e-06))
```

Turtlebot3 with DQN

- Experiments



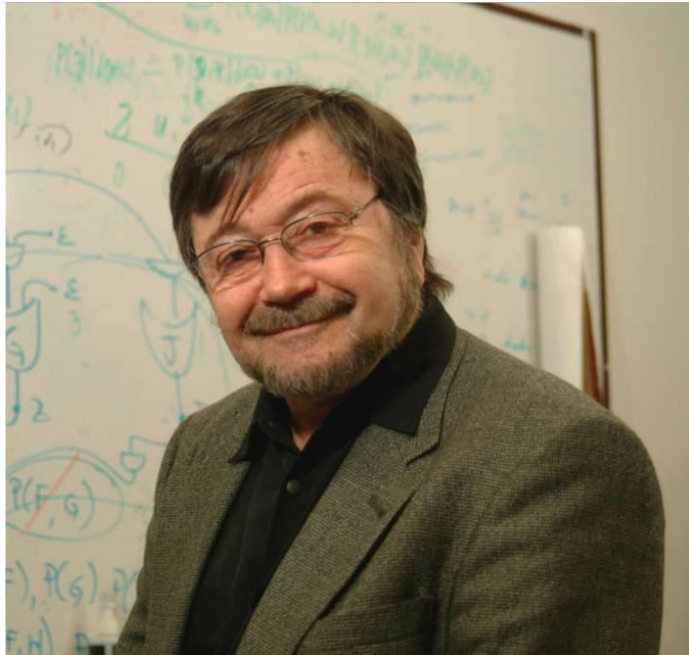
<https://www.youtube.com/watch?v=5uIZU8PCHT8&feature=youtu.be>

Conclusion

- DL and DRL
- 😊:
 - Image pattern input
 - End-to-end approaches
- 😞:
 - Convergence and optimality (multiple layers → non convex)
 - Overfitting (too many parameters. it can be alleviated by L1 and dropout)
 - Large training samples

Conclusion

All the impressive achievements of deep learning amount to just curve fitting.



CSAIL - MIT ·

Ladies, if he:

- requires lots of supervision
- yet always wants more power
- can't explain decisions - optimizes for the average outcome
- dismisses problems as edge cases
- forgets things catastrophically

He's not your man, he's a deep neural network.

credit: Alex J. Champandard

Judea Pearl, Turing Award winner (invention of Bayesian networks)

Conclusion

- It's difficult for DL to play these games
- These games are not just action games! The deep neural networks cannot feel horrible or have nightmares.
- If you are interested in AI history & ChatGPT, check my talk.
<https://www.youtube.com/watch?v=I932ql8Nci0>



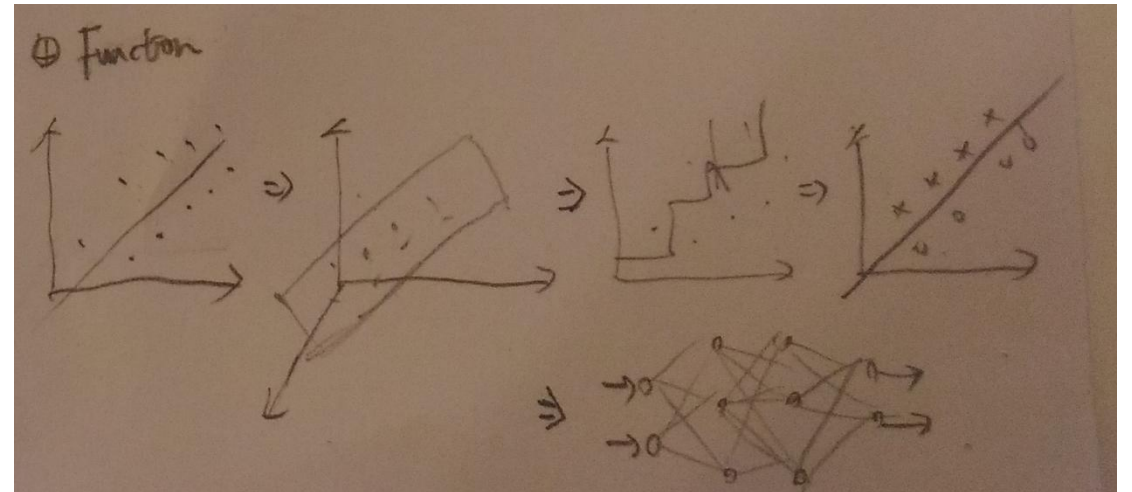
<https://en.wikipedia.org/wiki/Drakkhen>

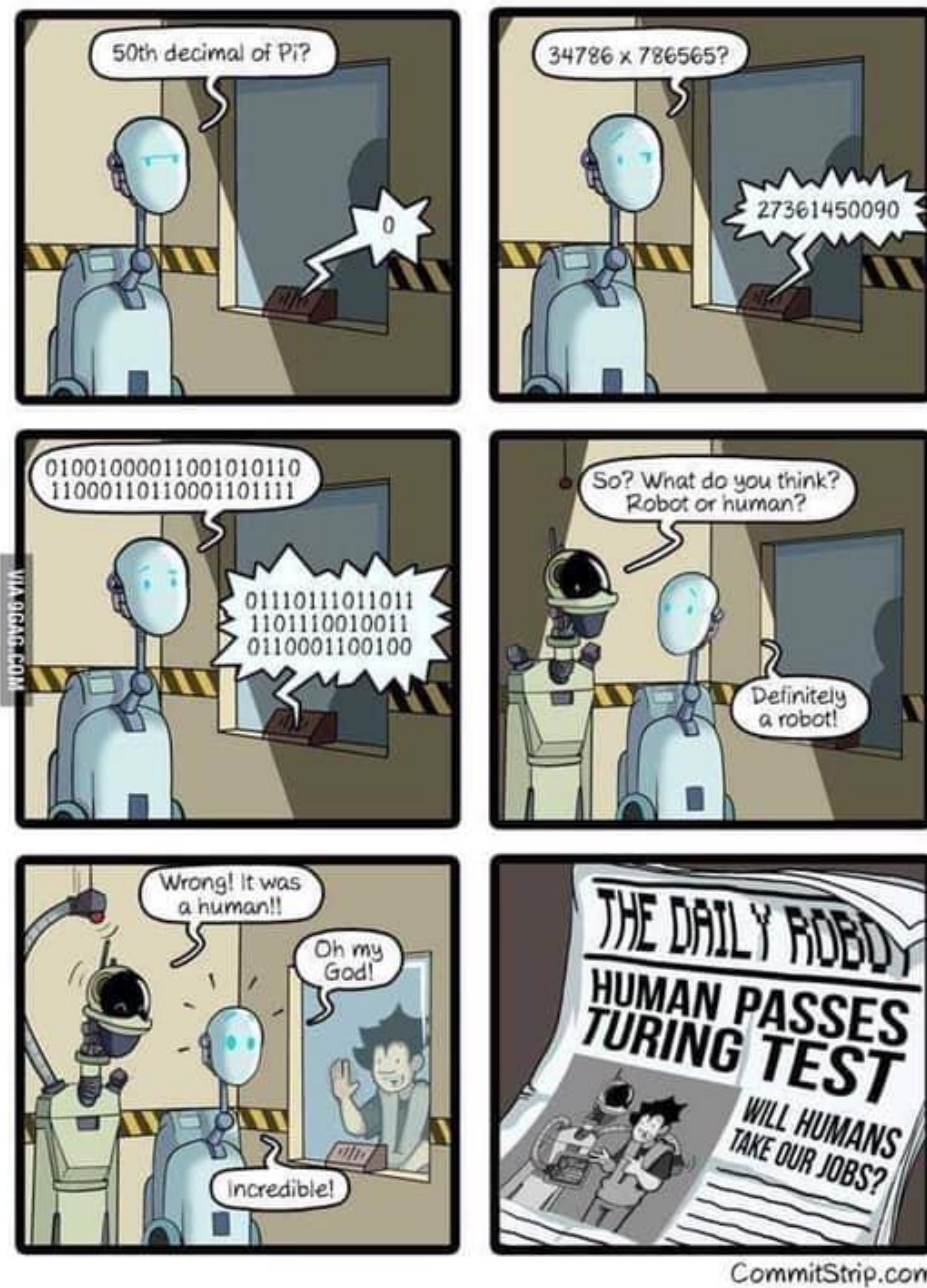


[https://en.wikipedia.org/wiki/Detention_\(video_game\)](https://en.wikipedia.org/wiki/Detention_(video_game))

Conclusion

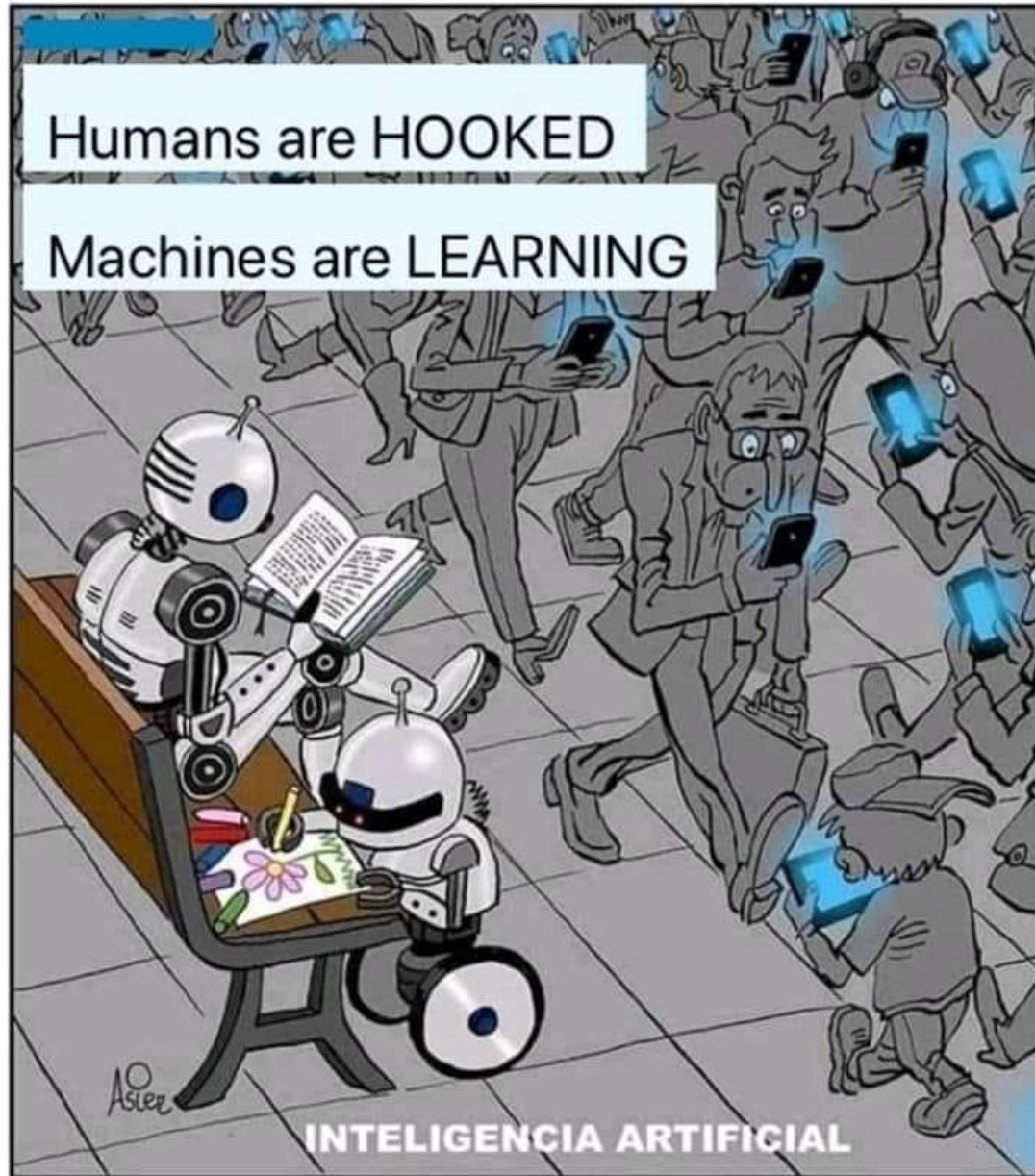
- As we mentioned, AI is to challenge Gods! Humans try to create new creatures.
- Deep reinforcement learning is the most promising AI approach so far.
- However, both “deep” and “reinforcement learning” were inspired by humans and animals.
- Supervised learning review:
 - Naïve Bayes
 - Perceptron, Logistic regression
 - Adaboost
 - Deep learning





Humans are HOOKED

Machines are LEARNING



INTELIGENCIA ARTIFICIAL

Q&A

